

Machine Learning

Roberto Carlos Santos Alonzo

14 de febrero de 2026

Índice

1. Matemáticas esenciales	3
1.1. Álgebra Lineal	3
1.1.1. Determinantes	3
2. Introducción	3
2.1. Flujo típico de trabajo de ML	3
2.2. Conjuntos de entrenamiento y prueba	6
2.2.1. Conjunto de entrenamiento	7
2.2.2. Conjunto de prueba	7
2.3. Tipos de Muestreo en Machine Learning	8
2.3.1. Muestreo Aleatorio (Random Sampling)	8
2.3.2. Muestreo Estratificado (Stratified Sampling)	8
2.3.3. Muestreo Ponderado (Weighted Sampling)	9
2.3.4. Muestreo por Importancia (Importance Sampling)	9
2.3.5. Muestreo de Yacimientos (Reservoir Sampling)	9
2.3.6. Resumen Comparativo	9
2.3.7. Sesgo de espionaje de datos (Data Leakage)	9
2.4. División de conjuntos de datos con Scikit-Learn	11
2.4.1. División aleatorio simple.	11
2.4.2. División estratificada	12
2.4.3. División en tres subconjuntos	12
2.5. Problemas comunes	13
2.5.1. Datos de entrenamiento no representativo	13
2.5.2. Sobreajuste (Overfitting)	14
2.5.3. Subajuste (Underfitting)	14
2.5.4. Datos de mala calidad	15
2.5.5. Características irrelevantes	16
2.6. Escalado y transformación de características.	16
2.6.1. Primeros pasos	16
2.6.2. Normalización (Min-Max Scaling)	20
2.6.3. Estandarización (Z-score Scaling)	22
2.6.4. Escalado logarítmico	25
2.6.5. Recorte	27
2.6.6. Robust Scaler	29
2.6.7. Conclusiones	31

2.6.8.	Aplicación en Python	33
2.7.	Escalado de variables Categoricalas	35
2.7.1.	Codificación Nominal (One-Hot)	36
2.7.2.	Cruces de características	39
2.7.3.	Codificación Ordinal	40
2.8.	Valores faltantes	43
2.8.1.	Simple Imputer	46
2.8.2.	Uso dentro de un Pipeline	49
2.9.	Análisis Exploratorio de Datos (EDA) para Machine Learning	50
2.10.	Medidas de forma	51
2.10.1.	Cómo calcular la asimetría en Python	52

1 Matemáticas esenciales

1.1 Álgebra Lineal

1.1.1. Determinantes

Definición 1.1 (Menor). Sea A una matriz de $n \times n$ y sea M_{ij} la matriz de $(n-1) \times (n-1)$ que se obtiene de A eliminando el renglón i y la columna j .

La matriz M_{ij} se llama el menor ij de A .

Definición 1.2 (Cofactor). Sea A una matriz de $n \times n$. El cofactor ij de A , denotado por A_{ij} , está dado por

$$A_{ij} = (-1)^{i+j} M_{ij} \quad (1.1)$$

Esto es, el cofactor ij de A se obtiene tomando el determinante del menor ij y multiplicándolo por $(-1)^{i+j}$. Observe que

$$(-1)^{i+j} = \begin{cases} 1, & \text{si } i+j \text{ es par,} \\ -1, & \text{si } i+j \text{ es impar.} \end{cases}$$

Definición 1.3 (Determinante $n \times n$). Sea A una matriz de $n \times n$. Entonces el determinante de A , denotado por $\det A$ o $|A|$, está dado por

$$\det A = |A| = a_{11}A_{11} + a_{12}A_{12} + \dots + a_{1n}A_{1n} = \sum_{k=1}^n a_{1k}A_{1k} \quad (1.2)$$

La expresión en el lado derecho de la ecuación (1.2) se llama *expansión por cofactores*.

2 Introducción

2.1 Flujo típico de trabajo de ML

1. Definir el problema y métrica:

Un proyecto de Aprendizaje Automático empieza por **enunciar con precisión el problema y la métrica de éxito**. Conocer el objetivo es importante, ya que determinará como enmarcará el problema, que algoritmos seleccionará, que medida de rendimiento utilizará para evaluar su modelo y cuanto esfuerzo dedicará a ajustarlo.

- No es lo mismo predecir si un correo es *spam* que estimar el precio de una casa.

2. Preparar los datos:

Con el objetivo claro, pasamos a **preparar los datos**.

- **Ingesta de datos:** recolección desde distintas fuentes como bases de datos, APIs, archivos CSV o sensores.
- **Preprocesamiento o limpieza:** eliminación de valores nulos, duplicados o inconsistentes (atípicos), y normalización de escalas numéricas.

- **Transformación:** conversión de formatos, codificación de variables categóricas y creación de nuevas características (*feature engineering*).
- **División de datos:** separación del conjunto en entrenamiento, validación y prueba.
- **Entrenamiento del modelo:** el modelo recibe los datos preparados y se ajusta a ellos.
- **Evaluación y despliegue:** el modelo se evalúa y, si cumple con los requisitos, se implementa en producción.

Este paso se vuelve sólido cuando se encapsula en **pipelines reproducibles**, de modo que las transformaciones sean coherentes en todo momento.

Una secuencia de componentes de procesamiento de datos se denomina **canalización de datos**. Las canalizaciones son muy comunes en los sistemas de aprendizaje automático, ya que hay una gran cantidad de datos que manipular y muchas transformaciones que aplicar.

3. Entrenar una línea base:

Antes de sofisticar el modelo, conviene entrenar una **línea base**. Una línea base es un modelo inicial y simple que sirve como punto de referencia para evaluar el rendimiento de modelos más complejos.

Por ejemplo, en tareas de clasificación, se puede emplear un *clasificador dummy* que siempre prediga la clase más frecuente o que realice predicciones al azar siguiendo la distribución de las clases. En problemas de regresión, la línea base podría ser un modelo que siempre prediga el promedio o la mediana del valor objetivo.

Entrenar una línea base cumple varias funciones importantes:

- **Diagnóstico del problema:** si un modelo complejo apenas mejora los resultados de la línea base, el problema suele estar en los *datos* o en la *formulación del problema*, no en la arquitectura del modelo.
- **Medición del progreso:** permite establecer un punto de comparación para medir mejoras reales en los modelos sucesivos.
- **Eficiencia:** evita invertir tiempo y recursos en modelos sofisticados cuando aún no se ha validado la calidad de los datos o la viabilidad de la tarea.

Por ejemplo, si se desarrolla un modelo para predecir si un cliente realizará una compra:

- a) Se entrena un *clasificador dummy* que siempre predice “no compra”, obteniendo un 70 % de exactitud.
- b) Luego, se entrena un modelo más complejo (por ejemplo, un árbol de decisión) y se obtiene un 72 %.
- c) La mejora es mínima, lo cual sugiere revisar los datos o la definición del problema antes de aumentar la complejidad del modelo.

En resumen, la línea base proporciona un **piso creíble de comparación** que permite validar la calidad de los datos y orientar los esfuerzos de modelado de manera más efectiva.

4. Validar y ajustar:

Usamos **validación cruzada** y búsqueda de **hiperparámetros** para expresar el modelo sin memorizar el conjunto de validación. Aquí es crucial vigilar la **fuga de datos**: todas las transformaciones (imputaciones, escalados, selección de variables) deben ajustarse solo con los datos de entrenamiento y aplicarse después al resto.

Una vez entrenado el modelo inicial, el siguiente paso es **validar y ajustar**. Este proceso busca evaluar el rendimiento del modelo en datos no vistos y optimizar sus hiperparámetros para mejorar su capacidad de generalización.

- **Validar:** consiste en medir el desempeño del modelo sobre un conjunto de datos diferente al de entrenamiento. Para ello se utilizan técnicas como la *validación cruzada* (*cross-validation*), que permiten estimar de forma más fiable el error de generalización y detectar posibles problemas de sobreajuste (*overfitting*).
- **Ajustar:** implica modificar los **hiperparámetros** del modelo —por ejemplo, la profundidad de un árbol de decisión, la tasa de aprendizaje de un modelo de boosting o los coeficientes de regularización— con el fin de optimizar su rendimiento. Entre las estrategias comunes se incluyen *Grid Search*, *Random Search* y *Optimización Bayesiana*.

El objetivo final de esta etapa es encontrar un modelo que no solo obtenga buenos resultados en los datos de entrenamiento, sino que también **generalice correctamente** a datos nuevos.

5. Evaluar en el conjunto de prueba:

El **conjunto de prueba** se deja intacto hasta el final. Es la fotografía más cercana a como rendirá el sistema con datos nuevos. Se evalúa una sola vez para estimar el desempeño fuera de muestra.

Una vez seleccionado el mejor modelo tras el proceso de validación y ajuste, se procede a **evaluarlo en el conjunto de prueba** (*test set*). Este conjunto contiene datos completamente nuevos, no utilizados durante el entrenamiento ni la validación, y su objetivo es estimar el rendimiento real del modelo en situaciones futuras o en producción.

- Se utiliza el modelo final (con los hiperparámetros óptimos) para generar predicciones sobre el conjunto de prueba.
- Se calculan las métricas finales de desempeño, tales como precisión, F1-score, AUC, error cuadrático medio (RMSE), entre otras.
- Se comparan los resultados con la **línea base** y los objetivos del proyecto para determinar si el modelo cumple con los requisitos esperados.

Es importante destacar que el conjunto de prueba debe permanecer **intacto** hasta esta etapa. Usarlo durante el entrenamiento o la validación introduciría sesgos y produciría estimaciones optimistas del rendimiento. La evaluación en el test set proporciona, por tanto, una medida honesta de la **capacidad de generalización** del modelo.

6. Despliegue y monitoreo:

Se versiona el modelo y el **pipeline**, se guarda el artefacto (y su *environment*), y se instrumentan tableros para vigilar *drift* de datos, rendimiento por segmentos y tasas de error. El mantenimiento es parte del producto: los datos cambian, y el modelo debe adaptarse con *retraining* controlado.

Una vez que el modelo ha sido validado y evaluado en el conjunto de prueba, se procede a su **despliegue en un entorno de producción**. Esto implica integrar el modelo en una aplicación, servicio o sistema donde pueda generar predicciones en tiempo real o por lotes.

- **Despliegue:** consiste en trasladar el modelo desde el entorno de desarrollo a producción, garantizando su reproducibilidad, seguridad y eficiencia. Se pueden utilizar herramientas como *Docker*, *MLflow* o *Kubeflow* para gestionar entornos, versiones y dependencias. También se asegura que el modelo cumpla con los requisitos de latencia, escalabilidad y mantenimiento.
- **Monitoreo:** una vez en producción, el modelo debe ser supervisado de forma continua para detectar posibles degradaciones en su desempeño. Esto incluye el seguimiento de métricas clave, la detección de cambios en la distribución de los datos (*data drift*) o en la relación entre variables y etiquetas (*concept drift*), y la programación de reentrenamientos periódicos si es necesario.

El objetivo de esta etapa es asegurar que el modelo mantenga su **rendimiento, confiabilidad y vigencia** a lo largo del tiempo, adaptándose a posibles cambios en los datos o en el contexto operativo.

2.2 Conjuntos de entrenamiento y prueba

El **Aprendizaje Automático** (Machine Learning, ML) es una rama de la inteligencia artificial que permite a los sistemas aprender patrones a partir de datos sin ser programados explícitamente.

Para entrenar y evaluar estos modelos, se utilizan diferentes subconjuntos del conjunto de datos original. Estos conjuntos tienen un papel fundamental en garantizar que el modelo no solo aprenda los patrones del conjunto de datos, sino que también sea capaz de generalizar a datos nuevos.

División del conjunto de datos

En la práctica, un conjunto de datos se divide típicamente en tres subconjuntos principales:

- **Conjunto de entrenamiento (Training set):** utilizado para ajustar los parámetros internos del modelo.
- **Conjunto de validación (Validation set):** empleado para seleccionar hiperparámetros y prevenir el sobreajuste.
- **Conjunto de prueba (Test set):** reservado para evaluar el rendimiento final del modelo con datos completamente nuevos.

La siguiente tabla resume las proporciones y propósitos típicos:

Conjunto	Propósito	Tamaño típico	Uso principal
Entrenamiento	Ajustar parámetros del modelo	60–80 %	Enseñar al modelo los patrones de los datos
Validación	Seleccionar hiperparámetros y prevenir sobreajuste	10–20 %	Evaluar desempeño durante el entrenamiento
Prueba	Evaluar rendimiento final del modelo	10–20 %	Medir la capacidad de generalización a datos nuevos

Cuadro 2.1: División típica de los conjuntos de datos en Machine Learning

2.2.1. Conjunto de entrenamiento

El conjunto de entrenamiento es el núcleo del proceso de aprendizaje. Durante esta etapa, el algoritmo analiza las variables de entrada (*features*) y ajusta sus parámetros internos para minimizar un error o función de pérdida.

Formalmente, si el modelo se representa como una función $f_{\theta}(X)$ con parámetros θ , el entrenamiento consiste en encontrar los valores de θ que minimicen el error sobre el conjunto de entrenamiento:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

donde L es la función de pérdida (por ejemplo, error cuadrático medio o entropía cruzada).

2.2.2. Conjunto de prueba

El conjunto de prueba se utiliza **únicamente al final del entrenamiento** para evaluar la capacidad del modelo de generalizar a datos no vistos.

El rendimiento sobre este conjunto es una estimación del error de generalización:

$$E_{\text{test}} = \frac{1}{M} \sum_{j=1}^M L(f_{\theta^*}(x_j^{(\text{test})}), y_j^{(\text{test})})$$

Un modelo con buen desempeño en el conjunto de prueba se considera bien generalizado.

Esta separación debe hacerse antes de entrenar el modelo, y debe mantenerse inviolable: jamás se deben usar los datos de prueba para ajustar el modelo. Sólo así podemos obtener una estimación honesta de su capacidad de generalización.

2.3 Tipos de Muestreo en Machine Learning

En Machine Learning, la forma en que seleccionamos los datos de entrenamiento y prueba es crucial para obtener modelos representativos y eficientes. A continuación se describen los principales métodos de muestreo, sus aplicaciones y ejemplos prácticos.

2.3.1. Muestreo Aleatorio (Random Sampling)

- **Definición:** Selección de ejemplos de forma completamente aleatoria. Esto generalmente es adecuado si el conjunto de datos es lo suficientemente grande (especialmente en relación con el número de atributos), pero si no lo es, se corre el riesgo de introducir un **sesgo de muestreo significativo**.
- **Uso:** Cuando los datos están balanceados y el dataset es suficientemente grande.
- **Ventaja:** Fácil de implementar y rápido.
- **Desventaja:** Puede generar conjuntos de entrenamiento o prueba no representativos si las clases están desbalanceadas.
- **Ejemplo en ML:** Seleccionar aleatoriamente 80 % de imágenes de un dataset de dígitos manuscritos (MNIST) para entrenamiento.

2.3.2. Muestreo Estratificado (Stratified Sampling)

- **Definición:** Se seleccionan los ejemplos preservando la proporción de clases o categorías en cada subconjunto. La población se divide en subgrupos homogéneos llamados **estratos**, y se muestrea el número adecuado de casos de cada estrato para garantizar que el conjunto de prueba sea representativo de la población general.
- **Uso:** Datos desbalanceados, cuando se desea que la distribución de clases en entrenamiento y prueba sea la misma que en el dataset completo.
- **Ventaja:** Garantiza representatividad de cada clase.
- **Desventaja:** Más complejo de implementar que el muestreo aleatorio.
- **Ejemplo en ML:** División de un dataset de cáncer (benigno/maligno) para que entrenamiento y prueba mantengan la proporción original de casos malignos.

2.3.3. Muestreo Ponderado (Weighted Sampling)

- **Definición:** Cada ejemplo tiene un peso que influye en su probabilidad de ser seleccionado.
- **Uso:** Clases minoritarias o cuando algunas instancias son más importantes para el aprendizaje.
- **Ventaja:** Permite balancear la influencia de clases minoritarias.
- **Desventaja:** Pesos mal calibrados pueden introducir sesgo.
- **Ejemplo en ML:** Entrenamiento de un modelo de detección de fraude donde los casos de fraude reciben un peso mayor que las transacciones normales.

2.3.4. Muestreo por Importancia (Importance Sampling)

- **Definición:** Selección de ejemplos más informativos o relevantes para el aprendizaje.
- **Uso:** Aprendizaje activo o cuando se quiere priorizar ejemplos difíciles o raros.
- **Ventaja:** Reduce la cantidad de datos necesarios para entrenar.
- **Desventaja:** Requiere un análisis previo o un modelo inicial.
- **Ejemplo en ML:** Elegir ejemplos donde un modelo de clasificación de texto tiene mayor incertidumbre para re-etiquetado y mejora iterativa.

2.3.5. Muestreo de Yacimientos (Reservoir Sampling)

- **Definición:** Técnica para mantener una muestra uniforme de un flujo de datos grande que no cabe en memoria.
- **Uso:** Datos en tiempo real o datasets muy grandes.
- **Ventaja:** Permite mantener una muestra representativa de un flujo de datos continuo.
- **Desventaja:** No necesario para datasets pequeños.
- **Ejemplo en ML:** Selección de una muestra representativa de tweets que llegan en tiempo real para entrenar un modelo de análisis de sentimiento.

2.3.6. Resumen Comparativo

2.3.7. Sesgo de espionaje de datos (Data Leakage)

El **sesgo de espionaje de datos** ocurre cuando información del conjunto de prueba o del futuro se filtra al conjunto de entrenamiento, haciendo que el modelo aprenda patrones que no estarán disponibles al momento de predecir en situaciones reales. Esto puede generar métricas de desempeño artificialmente altas.

Tipo	Uso típico	Ventaja	Ejemplo en ML
Aleatorio	Datos balanceados	Simple	Selección aleatoria de imágenes MNIST para entrenamiento
Estratificado	Clases desbalanceadas	Representativo	División de dataset de cáncer manteniendo proporción benigno/maligno
Ponderado	Clases minoritarias / priorización	Mejora aprendizaje minorías	Detección de fraude con pesos mayores para transacciones fraudulentas
Importancia	Aprendizaje activo / muestras informativas	Reduce datos necesarios	Selección de textos con alta incertidumbre para re-etiquetado
Yacimientos	Datos en flujo / muy grandes	Útil en streaming	Muestra representativa de tweets en tiempo real

Cuadro 2.2: Comparación de métodos de muestreo en Machine Learning con ejemplos aplicados

Causas comunes:

- **Variables del futuro:** Incluir columnas que solo estarían disponibles después del evento objetivo.
- **Preprocesamiento inapropiado:** Escalar o normalizar usando todo el dataset en lugar de solo el conjunto de entrenamiento.
- **División de datos incorrecta:** Mezclar ejemplos altamente correlacionados entre entrenamiento y prueba, como en series temporales o datos de la misma fuente.

Ejemplo en Machine Learning:

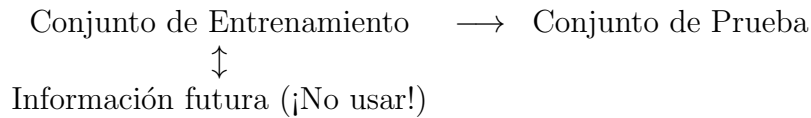
- Predicción de fraude: usar la columna “marcado como fraude” que solo se sabe después de la transacción.
- Predicción médica: usar resultados de laboratorio obtenidos tras el diagnóstico como características.

Prevención:

- Respetar la secuencia temporal o grupos independientes al dividir datos.

- Ajustar transformaciones y selección de características únicamente en el conjunto de entrenamiento.
- Revisar que ninguna variable contenga información del target futura.

Esquema visual sencillo:



La flecha horizontal representa el flujo normal de entrenamiento a prueba, mientras que la flecha vertical ilustra la **fuga de información** desde el futuro hacia el entrenamiento.

2.4 División de conjuntos de datos con Scikit-Learn

Scikit-Learn proporciona varias funciones para dividir conjuntos de datos en múltiples subconjuntos de distintas maneras. La función más simple y utilizada es `train_test_split()`, que permite dividir un dataset en subconjuntos de *entrenamiento* y *prueba*.

2.4.1. División aleatorio simple.

La función `train_test_split()` se importa desde `sklearn.model_selection` y acepta, entre otros, los siguientes parámetros:

- `test_size`: proporción o número absoluto de muestras para el conjunto de prueba.
- `train_size`: proporción o número absoluto de muestras para el conjunto de entrenamiento.
- `random_state`: semilla para garantizar reproducibilidad.

```

1  from sklearn.model_selection import train_test_split
2  import numpy as np
3
4  # Creamos un conjunto de datos de ejemplo
5  X = np.arange(10).reshape((5, 2)) # 5 muestras, 2
   características
6  y = np.array([0, 1, 0, 1, 0])
7
8  # División 80% entrenamiento, 20% prueba
9  X_train, X_test, y_train, y_test = train_test_split(
10 X, y, test_size=0.2, random_state=42
11 )
  
```

2.4.2. División estratificada

En problemas de clasificación, es importante que la proporción de clases se mantenga tanto en el conjunto de entrenamiento como en el de prueba. Esto se logra utilizando el parámetro `stratify`.

```

1      from sklearn.model_selection import train_test_split
2      from sklearn.datasets import load_iris
3      import numpy as np
4
5      iris = load_iris()
6      X = iris.data
7      y = iris.target
8
9      # División estratificada
10     X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.3, stratify=y, random_state=42
12     )
13
14     print("Proporciones en y original:", np.bincount(y) /
15           len(y))
16     print("Proporciones en y_train:", np.bincount(y_train
17           ) / len(y_train))
18     print("Proporciones en y_test:", np.bincount(y_test)
19           / len(y_test))

```

2.4.3. División en tres subconjuntos

En ocasiones es necesario crear tres conjuntos: *entrenamiento*, *validación* y *prueba*. Esto puede hacerse aplicando `train_test_split()` de manera encadenada.

```

1      from sklearn.model_selection import train_test_split
2      from sklearn.datasets import load_iris
3
4      iris = load_iris()
5      X = iris.data
6      y = iris.target
7
8      # Primera division: train+val y test
9      X_temp, X_test, y_temp, y_test = train_test_split(
10     X, y, test_size=0.2, random_state=42
11     )
12
13     # Segunda division: train y val
14     X_train, X_val, y_train, y_val = train_test_split(
15     X_temp, y_temp, test_size=0.25, random_state=42
16     )
17     # Resultado: 60% train, 20% val, 20% test
18
19     print(f"Entrenamiento: {len(X_train)} muestras")
20     print(f"Validacion: {len(X_val)} muestras")
21     print(f"Prueba: {len(X_test)} muestras")

```

Resumen de parámetros

Parámetro	Descripción
<code>test_size</code>	Proporción o número absoluto de muestras destinadas al conjunto de prueba.
<code>train_size</code>	Proporción o número absoluto de muestras destinadas al conjunto de entrenamiento.
<code>random_state</code>	Controla la aleatoriedad del muestreo para asegurar resultados reproducibles.
<code>stratify</code>	Garantiza que las proporciones de clases se mantengan en ambos subconjuntos.

Conclusión

La función `train_test_split()` constituye una herramienta fundamental para la preparación de datos en proyectos de *machine learning*, ya que permite dividir de forma sencilla y controlada los conjuntos de datos para entrenamiento, validación y prueba, manteniendo las proporciones de clases cuando es necesario.

2.5 Problemas comunes

2.5.1. Datos de entrenamiento no representativo

Para generalizar bien, es fundamental que los datos de entrenamiento sean representativos de los nuevos casos a los que se desea generalizar. Al utilizar un conjunto de entrenamiento no representativo, se entrena un modelo que probablemente no realice predicciones precisas.

Es fundamental utilizar un conjunto de entrenamiento que sea representativo de los casos a los que desea generalizar. Esto suele ser más difícil de lo que parece:

- Si la muestra es demasiado pequeña, se producirá **ruido de muestreo** (es decir, datos no representativos debido al azar).

Por ejemplo, si entrenamos un modelo de clasificación de imágenes para reconocer perros y gatos con solo 5 imágenes por clase, el modelo podría aprender características irrelevantes de esas imágenes específicas en lugar de patrones generales de cada animal.

- Si la muestra es grande pero el método de muestreo tiene fallas, los datos pueden ser no representativos. Esto se denomina **sesgo de muestreo**.

Por ejemplo, si entrenamos un modelo de predicción de riesgo crediticio usando solo datos de clientes de un banco en una ciudad específica, el modelo puede no generalizar bien a clientes de otras regiones con comportamientos financieros distintos.

- Otro ejemplo común es el **sesgo de clase** en problemas de clasificación desbalanceada. Por ejemplo, en un conjunto de datos de detección de fraude bancario donde el 99% de las transacciones son legítimas, un modelo podría predecir siempre "legítimo" aun así tener alta exactitud, fallando completamente al generalizar para detectar fraudes reales.

En Machine Learning, asegurar la representatividad de los datos de entrenamiento es clave para reducir errores de generalización y evitar que el modelo aprenda patrones espurios.

2.5.2. Sobreajuste (Overfitting)

Ocurre cuando el modelo memoriza los datos de entrenamiento en lugar de aprender los patrones subyacentes. En este caso, el error en el conjunto de entrenamiento es bajo, pero el error en el conjunto de prueba es alto.

Aparece cuando el modelo es demasiado complejo y se adhiere no solo a patrones reales de los datos, sino también al ruido a las peculiaridades accidentales del conjunto de entrenamiento. En este caso, $E_{train}(\hat{f})$ es muy bajo (el modelo parece perfecto sobre los datos ya vistos), pero $E_{test}(\hat{f})$ crece significativamente porque $\hat{f}$ no generaliza bien a datos nuevos.

$$E_{train} \ll E_{test}$$

Ejemplo:

- Un alumno se limitó a memorizar el cuestionario. Si el examen contiene exactamente las mismas preguntas, parecerá sobresaliente. Pero basta con que el examen introduzca ligeras variaciones para que quede en evidencia que no domina el tema, sino que solo repitió mecánicamente lo que había aprendido.

Soluciones: Regularización, más datos, validación cruzada, reducción de la complejidad del modelo.

- Simplifique el modelo seleccionado uno con menos **parámetros** (por ejemplo, un modelo lineal en lugar de un modelo polinomial de alto grado), reduciendo la cantidad de atributos en los datos de entrenamiento o restringiendo el modelo.
- Recopilar más datos de entrenamiento.
- Reducir el ruido de los datos de entrenamiento (por ejemplo, corregir errores de datos y eliminar valores atípicos).
- Restringir un modelo para simplificarlo y reducir el riesgo de **sobreajuste** se denomina **regularización**.

2.5.3. Subajuste (Underfitting)

Sucede cuando el modelo no logra capturar la relación entre las variables de entrada y salida. Tanto el error de entrenamiento como el de prueba son altos.

Este ocurre cuando el modelo es demasiado limitado para captar la estructura de los datos. En términos matemáticos, si f es la función que queremos aproximar y $\hat{f}$ es la función aprendida, el error de entrenamiento $E_{train}(\hat{f})$ y el error de prueba $E_{test}(\hat{f})$ son ambos altos. Es decir, $\hat{f}$ no logra representar ni siquiera lo que ya vio en el conjunto de entrenamiento.

$$E_{\text{train}} \approx E_{\text{test}} \text{ (ambos altos)}$$

Ejemplo:

- Un alumno estudió poco y de manera superficial. En este caso no logra responder ni las preguntas que venían en el cuestionario ni aquellas que se parecen, porque nunca llegó a comprender con suficiente profundidad la materia.

Soluciones: Utilizar un modelo más complejo, añadir características relevantes o entrenar durante más iteraciones.

- Seleccione un modelo más potente, con más parámetros.
- Alimente con mejores características el algoritmo de aprendizaje (ingeniería de características).
- Reducir las restricciones del modelo (por ejemplo, reduciendo el **hiperparámetro de regularización**)

El reto central de aprendizaje automático, consiste en encontrar un modelo con la complejidad adecuada para que el error de entrenamiento sea razonablemente bajo sin que el error de prueba se dispare. A ese equilibrio se suele llama simplemente **buen ajuste** o **generalización adecuada**.

2.5.4. Datos de mala calidad

Si sus datos de entrenamiento están llenos de errores, **valores atípicos y ruido** (por ejemplo, debido a mediciones de baja calidad), al sistema le resultará más difícil detectar los patrones subyacentes, por lo que es menos probable que funcione bien.

Ejemplo:

- El alumno realmente se esforzó, dedicó tiempo y estudió con seriedad, pero el cuestionario que recibió para prepararse estaba mal hecho: contenía errores, ejemplos pocos representativos o temas que no correspondían a la materia real. En ese caso, el bajo desempeño en el examen no refleja su falta de dedicación, sino la mala calidad del material con el que se preparó.

A continuación se muestran algunos ejemplos de cuando conviene depurar los datos de entrenamiento:

- Si algunos casos son claramente atípicos, puede ser útil simplemente descartarlos o intentar corregir los errores manualmente.
- Si en algunas instancias faltan algunas características (por ejemplo, el 5 % de sus clientes no especificaron su edad), debe decidir si desea ignorar este atributo por completo, ignorar estas instancias, completar los valores faltantes (por ejemplo, con la edad media) o entrenar un modelo con la característica y un modelo sin ella.

2.5.5. Características irrelevantes

Como dice el dicho: si entra basura, sale basura. Un aspecto fundamental del éxito de un proyecto de Machine Learning es crear un buen conjunto de características con las que entrenar. Este proceso, llamado **ingeniería de características**, implica los siguientes pasos:

1. **Selección de características:** Seleccionar las características más útiles para entrenar las características existentes.
2. **Extracción de características** (combinar características existentes para producir una más útil).
3. Creación de nuevas funciones mediante la recopilación de nuevos datos.

2.6 Escalado y transformación de características.

2.6.1. Primeros pasos

Antes de crear vectores de características, te recomendamos que estudies los datos numéricos de estas dos maneras:

Visualiza tus datos en gráficos o diagramas.

- Obtén estadísticas sobre tus datos.
- Visualiza tus datos

Visualiza tus datos

Los gráficos pueden ayudarte a encontrar anomalías o patrones ocultos en los datos. Por lo tanto, antes de avanzar demasiado en el análisis, observa tus datos de forma gráfica, ya sea como diagramas de dispersión o histogramas. Consulta los gráficos no solo al comienzo de la canalización de datos, sino también durante las transformaciones de datos. Las visualizaciones te ayudan a verificar tus suposiciones de forma continua.

Te recomendamos trabajar con pandas para la visualización:

- Cómo trabajar con datos faltantes (Documentación de pandas)
- Visualizaciones (documentación de pandas)

Ten en cuenta que algunas herramientas de visualización están optimizadas para ciertos formatos de datos.

Evalúa tus datos de forma estadística.

Además del análisis visual, también recomendamos evaluar las posibles funciones y etiquetas de forma matemática y recopilar estadísticas básicas, como las siguientes:

- media y mediana.
- desviación estándar.
- los valores en las divisiones de cuartil: los percentiles 0, 25, 50, 75 y 100. El percentil 0 es el valor mínimo de esta columna, y el percentil 100 es el valor máximo de esta columna. (el percentil 50 es la mediana).

Cómo encontrar valores atípicos

Un **valor atípico** es un valor distante de la mayoría de los otros valores de un atributo o una etiqueta. Los valores atípicos suelen causar problemas en el entrenamiento del modelo, por lo que es importante encontrarlos.

Cuando la diferencia entre el percentil 0 y el 25 difiere significativamente de la diferencia entre el percentil 75 y el 100, es probable que el conjunto de datos contenga valores atípicos.

Nota: No dependas demasiado de las estadísticas básicas. Las anomalías también pueden ocultarse en datos que parecen estar bien equilibrados. Los valores atípicos pueden pertenecer a cualquiera de las siguientes categorías:

- El valor atípico se debe a un error. Por ejemplo, es posible que un experimentador haya ingresado por error un cero adicional o que un instrumento que recopiló datos haya fallado. Por lo general, borrarás los ejemplos que contengan valores extremos por errores.
- El valor atípico es un dato legítimo, no un error. En este caso, ¿tu modelo entrenado necesitará, en última instancia, inferir buenas predicciones sobre estos valores atípicos?
 - Si es así, mantén estos valores atípicos en tu conjunto de entrenamiento. Después de todo, los valores extremos de ciertas características a veces reflejan los valores extremos de la etiqueta, por lo que los valores extremos podrían ayudar a tu modelo a realizar mejores predicciones. Ten cuidado, los valores atípicos extremos aún pueden perjudicar tu modelo.
 - De lo contrario, borra los valores atípicos o aplica técnicas de ingeniería de atributos más invasivas, como el **recorte**.

Python - Análisis

```
1 import pandas as pd
2
3 # The following lines adjust the granularity of
4   reporting.
5 pd.options.display.max_rows = 10
6 pd.options.display.float_format = "{:.1f}".format
7 # The following code imports the dataset that is used
8   in the colab.
9
10 training_df = pd.read_csv(filepath_or_buffer="https
11   ://download.mlcc.google.com/mledu-datasets/
12   california_housing_train.csv")
13 # The following code returns basic statistics about
14   the data in the dataframe.
15
16 training_df.describe()
17 print("""The following columns might contain outliers
18   :
19   """)
```

```
14      * total_rooms
15      * total_bedrooms
16      * population
17      * households
18      * possibly, median_income
19
20      In all of those columns:
21
22      * the standard deviation is almost as high as the
23        mean
24      * the delta between 75% and max is much higher than
25        the
26        delta between min and 25%.")
```

Los profesionales del aprendizaje automático dedican mucho más tiempo a evaluar, limpiar y transformar datos que a construir modelos. ¿No produciría un modelo mejores predicciones si se entrenara con los valores reales del conjunto de datos en lugar de con los valores alterados? Sorprendentemente, la respuesta es no. Debes determinar la mejor manera de representar los valores del conjunto de datos sin procesar como valores entrenables en el **vector de características**. Este proceso se denomina **ingeniería de características** y es una parte fundamental del aprendizaje automático.

Una de las transformaciones más importantes que debe aplicar a sus datos es el **escalado de características**. La escala de características es un paso crucial en nuestro **pipeline** de preprocesamiento, que puede olvidarse fácilmente.

- Los árboles de decisión y los bosques aleatorios son dos de los pocos algoritmos de aprendizaje automático en los que no es necesario preocuparse por la escala de características. Estos algoritmos son invariables en cuanto a la escala.
- La mayoría de los algoritmos de aprendizaje automático y de optimización se comportan mucho mejor si las características están en la misma escala.

En Machine Learning no todas las variables hablan el mismo “idioma numérico”. Algunas se mueven en intervalos pequeños, como la edad de una persona entre 0 y 100 años, mientras que otras pueden alcanzar magnitudes enormes, como los ingresos anuales en millones. Si ambas entran juntas en un algoritmo sin ningún ajuste, la segunda tenderá a dominar la primera no porque sea más importante, sino simplemente porque sus números son más grandes. Para un modelo, una diferencia de 10,000 puede parecer infinitamente más relevante que una diferencia de 10, aunque en la realidad suceda lo contrario.

Este desbalance afecta especialmente a los algoritmos que dependen de distancias o gradiente.

- En el algoritmo de k-vecinos más cercanos (KNN), con una medida de distancia euclidiana: las distancias calculadas entre los ejemplos estarán dominadas por el segundo eje de características.
- En redes neuronales o en regresión logística, el problema aparece en la fase de optimización: las variables con magnitudes grandes generan gradientes más

pronunciados, y el algoritmo de entrenamiento avanza con pasos desproporcionados en unas direcciones y con pasos minúsculos en otras. El resultado puede ser una convergencia lenta e inestable.

Para evitar sesgos, recurrimos al **escalado de variables**, cuyo objetivo es ajustar las magnitudes sin alterar la información esencial. Escalar es un acto de justicia numérica: todas las variables pasan a jugar en un mismo campo, y es el algoritmo el que decidirá cuáles son realmente importantes según el patrón de los datos, no por el tamaño de las cifras.

Ejemplo ilustrativo:

1. Una buena forma de entender el escalado es imaginar una balanza. Si colocamos en un plato una canica de 3 gramos y en el otro una sandía de 3 kilogramos, la comparación queda anulada: *la sandía siempre ganará*.. El problema no es la irrelevancia de la canica, sino que ambas están en escalas distintas. El escalado es como usar una balanza calibrada que no compara pesos absolutos sino proporciones, devolviendo a la canica la posibilidad de ser tomada en cuenta.
2. Supongamos un conjunto de datos con dos variables: la edad x_1 y el ingreso anual x_2 (en dólares). Sea una muestra:

$$\mathbf{x} = [x_1, x_2] = [25, 50,000].$$

Si aplicamos una métrica euclidiana entre dos observaciones $(x_1^{(i)}, x_2^{(i)})$ y $(x_1^{(j)}, x_2^{(j)})$, tenemos:

$$d_{ij} = \sqrt{(x_1^{(i)} - x_1^{(j)})^2 + (x_2^{(i)} - x_2^{(j)})^2}.$$

Claramente, el término de x_2 dominará debido a su magnitud, haciendo que la diferencia en edad sea prácticamente irrelevante en la distancia total.

3. También podemos pensar en una orquesta desafinada. Imagina que cada instrumento toca con un volumen distinto: la trompeta suena tan fuerte que opaca al violín. El director no logra escuchar el balance real de la obra. El escalado es como ajustar el volumen de cada instrumento para que todos puedan ser oídos, y entonces sí, la melodía global se revela con claridad.

Considera los siguientes dos atributos:

- El valor más bajo de la característica A es -0,5 y el más alto es +0,5.
- El valor más bajo de la característica B es -5,0 y el más alto es +5,0.

Las características A y B tienen rangos relativamente estrechos. Sin embargo, el intervalo de la característica B es 10 veces más amplio que el de la característica A. Por lo tanto:

- Al comienzo del entrenamiento, el modelo supone que el atributo B es diez veces más importante " que el atributo A.

- El entrenamiento tardará más de lo que debería.
- El modelo resultante puede ser deficiente.

El daño general por no normalizar será relativamente pequeño. Sin embargo, recomendamos normalizar la característica A y la característica B en la misma escala, tal vez de -1,0 a +1,0.

Ahora, considera dos atributos con una mayor disparidad de rangos:

- El valor más bajo de la función C es -1 y el más alto es +1.
- El valor más bajo de la función D es +5,000 y el más alto es +1,000,000,000.

Si no normalizas el atributo C y el atributo D, es probable que tu modelo no sea óptimo. Además, el entrenamiento tardará mucho más en converger o incluso no podrá hacerlo.

Existen varios caminos para poner a las variables en una escala comparable. No todos sirven para lo mismo: cada método corresponde a un problema distinto y produce un efecto particular con los datos.

2.6.2. Normalización (Min-Max Scaling)

El escalamiento Min-Max (muchas personas lo llaman **normalización**) es el más simple: para cada atributo, los valores se desplazan y se reescalan para que terminen entre 0 y 1. Esto se realiza restando el valor mínimo y dividiéndolo por la diferencia entre el mínimo y el máximo. Para normalizar nuestros datos, podemos aplicar la escala mínima-máxima a cada columna de características, donde el nuevo valor, $x_{norm}^{(i)}$, de un ejemplo, $x^{(i)}$, se puede calcular como sigue:

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}} \Rightarrow x_{norm}^{(i)} \in [0, 1]. \quad (2.1)$$

Aquí, $x^{(i)}$ es un ejemplo particular, x_{min} es el menor valor de una columna de características, y x_{max} es el mayor valor.

La normalización es una buena opción cuando se cumplen todas las siguientes condiciones:

- Los límites inferior y superior de tus datos no cambian mucho con el tiempo.
- La característica contiene pocos o ningún valor atípico, y esos valores atípicos no son extremos.
- La característica se distribuye de forma aproximadamente uniforme en todo su rango. Es decir, un histograma mostraría barras prácticamente iguales para la mayoría de los valores.

Supongamos que la **edad** es una característica. El ajuste lineal es una buena técnica de normalización para **edad** porque:

Los límites inferior y superior aproximados son de 0 a 100. **edad** contiene un porcentaje relativamente pequeño de valores atípicos. Solo alrededor del 0.3 % de la

población tiene más de 100 años. Si bien ciertas edades están algo mejor representadas que otras, un conjunto de datos grande debería contener suficientes ejemplos de todas las edades.

La normalización es intuitiva y muy útil cuando los datos tienen **límites conocidos** o cuando se desea mantener la interpretación relativa de los valores. Es común en redes neuronales, donde es conveniente que los números estén en un rango acotado para favorecer la estabilidad numérica. Sin embargo, si aparece un nuevo valor fuera del rango visto en el entrenamiento, la transformación puede perder sentido: por ejemplo, un ingreso de 200,000 que no estaba presente anteriormente daría un valor superior a 1, lo que puede ser problemático.

- Es altamente sensible a **outliers**: un solo valor atípico puede “aplastar” a todos los demás hacia un rango muy estrecho, reduciendo la efectividad del escalamiento.
- Mantiene la **distribución relativa** de los datos, pero no afecta la forma de la distribución original (no transforma datos no lineales en lineales).
- Ideal para algoritmos basados en **distancias** (como *k-NN* o *Redes Neuronales*) y aquellos sensibles a la magnitud de las características.

Scikit-Learn proporciona un transformador llamado **MinMaxScaler** para realizar esta normalización de manera sencilla. Tiene un hiperparámetro **feature_range** que permite cambiar el rango de los valores normalizados. Por ejemplo, en redes neuronales es frecuente utilizar un rango de $[-1, 1]$ en lugar de $[0, 1]$ para centrar los datos alrededor de cero y mejorar la convergencia durante el entrenamiento:

Ejemplo

15,000 $\rightarrow$ 0
22,000 $\rightarrow$ 0,038
18,000 $\rightarrow$ 0,016
28,000 $\rightarrow$ 0,070
200,000 $\rightarrow$ 1
35,000 $\rightarrow$ 0,108
25,000 $\rightarrow$ 0,054

```
1 import pandas as pd
2 from sklearn.preprocessing import MinMaxScaler
3 valores = [15000, 22000, 18000, 28000, 200000,
4            35000, 25000]
5 df = pd.DataFrame(valores, columns = ['Valores'])
6 min_max_scaled = MinMaxScaler(feature_range = (0,1))
7 df_minmax_scaled = min_max_scaled.fit_transform(df)
df_minmax_scaled
```

La normalización proporciona los siguientes beneficios:

- Ayuda a que los modelos converjan más rápido durante el entrenamiento. Cuando las diferentes funciones tienen rangos diferentes, el descenso del gradiente puede rebotar o ralentizar la convergencia. Dicho esto, los optimizadores más avanzados, como Adagrad y Adam, protegen contra este problema cambiando la tasa de aprendizaje efectiva con el tiempo.
- Ayuda a los modelos a inferir mejores predicciones. Cuando las diferentes características tienen rangos diferentes, el modelo resultante podría generar predicciones algo menos útiles.
- Ayuda a evitar la "trampa de NaN" cuando los valores de las características son muy altos. **NaN** es la abreviatura de "no es un número". Cuando un valor de un modelo supera el límite de precisión de punto flotante, el sistema establece el valor en NaN en lugar de un número. Cuando un número del modelo se convierte en NaN, otros números del modelo también se convierten en NaN.
- Ayuda al modelo a aprender los pesos adecuados para cada atributo. Sin el ajuste de atributos, el modelo les presta demasiada atención a los atributos con rangos amplios y no les presta suficiente atención a los atributos con rangos estrechos.

Nota: Si normaliza una característica durante el entrenamiento, también debe normalizar esa característica al realizar predicciones.

Resumen de buenas prácticas:

- Verificar la presencia de **outliers** antes de aplicar Min-Max.
- Considerar rangos alternativos si se requiere media cero o simetría.
- Ajustar el transformador usando solo el conjunto de entrenamiento y aplicar la misma transformación al conjunto de prueba.

2.6.3. Estandarización (Z-score Scaling)

La **estandarización Z** indica cuántas desviaciones estándar se encuentra un valor respecto a su media. Por ejemplo, un valor que se encuentra a 2 desviaciones estándar por encima de la media tiene una puntuación Z de +2,0. Un valor que se encuentra a 1,5 desviaciones estándar por debajo de la media tiene una puntuación Z de -1,5.

- A la derecha, la misma distribución normalizada mediante el escalamiento Z .
- El escalamiento Z también es una buena opción para datos como los que se muestran en la siguiente figura, que tienen solo una distribución vagamente normal.

En una distribución normal clásica:

- Al menos el 68,27 % de los datos tiene una puntuación Z entre -1 y 1.

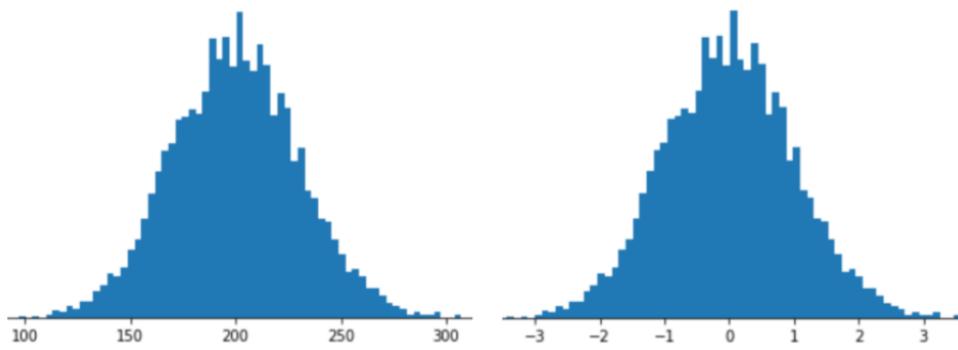


Figura 2.1 – *Datos brutos (izquierda) vs escalamiento Z (derecha) para una distribución normal.*

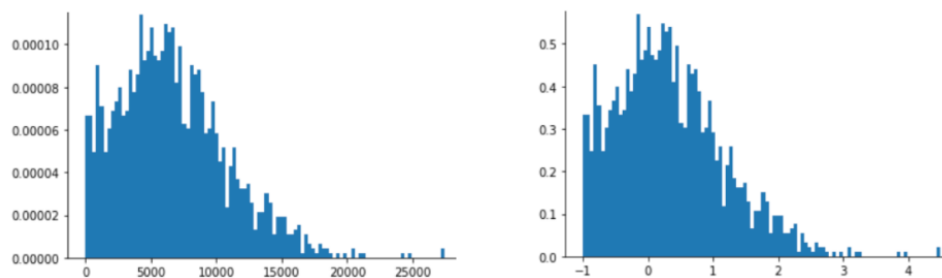


Figura 2.2 – *Datos sin procesar (izquierda) vs escalamiento Z (derecha) para una distribución normal no clásica.*

- Al menos el 95,45 % de los datos tiene una puntuación Z entre -2 y 2 .
- Al menos el 99,73 % de los datos tiene una puntuación Z entre -3 y 3 .
- Al menos el 99,994 % de los datos tiene una puntuación Z entre -4 y 4 .

Por lo tanto, los datos con una puntuación Z inferior a -4.0 o superior a $+4.0$ son poco frecuentes, pero ¿son realmente valores atípicos? Dado que los valores atípicos son un concepto sin una definición estricta, nadie puede afirmarlo con certeza. Ten en cuenta que un conjunto de datos con una cantidad suficientemente grande de ejemplos casi con certeza contendrá al menos algunos de estos ejemplos "poco frecuentes". Por ejemplo, un atributo con mil millones de ejemplos que se ajustan a una distribución normal clásica podría tener hasta 60,000 ejemplos con una puntuación fuera del rango de $-4,0$ a $+4,0$.

La puntuación Z es una buena opción cuando los datos siguen una distribución normal o una distribución algo similar a una distribución normal.

Ten en cuenta que algunas distribuciones pueden ser normales dentro de la mayor parte de su rango, pero aun así contener valores atípicos extremos. Por ejemplo, casi todos los puntos de un atributo **patrimonio** podrían ajustarse perfectamente a 3 desviaciones estándares, pero algunos ejemplos de este atributo podrían estar a cientos de desviaciones estándares de la media. En estas situaciones, puedes combinar el ajuste de la puntuación Z con otra forma de normalización (por lo general, el recorte) para controlar esta situación.

La estandarización puede ser más práctica para muchos algoritmos de aprendizaje automático, especialmente para los algoritmos de optimización como el **descenso del gradiente**. La razón es que muchos modelos lineales, como la **regresión logística** y **SVM**, inicializan los pesos a 0 o a pequeños valores aleatorios cercanos a 0.

Utilizando la estandarización, centramos las columnas de características en la media 0 con desviación estándar 1 para que estas tengan los mismos parámetros que una **distribución normal estándar** (media cero y varianza unitaria), lo que facilita la averiguación de los pesos.

- La estandarización no cambia la forma de la distribución y no transforma los datos no distribuidos normalmente en datos distribuidos normalmente..
- La estandarización conserva información útil sobre los valores atípicos y hace que el algoritmo sea menos sensible a ellos, en contraste con el escalado **min-máx**, que escala datos a un rango limitado de valores.

Primero se resta la media y luego se divide el resultado entre la desviación estándar. El procedimiento de estandarización puede expresarse mediante la siguiente ecuación.

$$x_{std}^i = \frac{x^i - \mu_x}{\sigma_x} \quad (2.2)$$

Aquí, μ_x es la **media muestral** de una columna de características concreta, y σ_x es la desviación estándar correspondiente.

- **Poblacional:** cuando se conocen todos los datos de la población:

$$\text{Media poblacional: } \mu_x = \frac{1}{N} \sum_{i=1}^N x_i,$$

$$\text{Desviación estándar poblacional: } \sigma_x = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}.$$

La ventaja de este método es que **respeto la forma de la distribución**: Si la variable era normal antes, lo seguirá siendo después, sólo que centrada en 0 y con varianza unitaria. Por eso se utiliza mucho en algoritmos sensibles a las varianzas relativas, como regresión logística, SVM o redes neuronales.

Su limitación es que resulta sensible a valores atípicos. Un solo dato extremadamente grande o pequeño puede alterar la media y la desviación estándar, distorsionando todo el escalado.

Ejemplo

Primero calculamos la media y la desviación estándar:

- Media $\mu_x = 49,000$; desviación estándar (considerando población) $\sigma_x \approx 61,945$
Aplicamos la fórmula:

$$x_{std}^i = \frac{x^i - \mu_x}{\sigma_x}$$

15,000 $\rightarrow$ -0,55
 22,000 $\rightarrow$ -0,44
 18,000 $\rightarrow$ -0,50
 28,000 $\rightarrow$ -0,34
 200,000 $\rightarrow$ +2,43
 35,000 $\rightarrow$ -0,23
 25,000 $\rightarrow$ -0,39

El valor atípico desplaza la media hasta 49,000 y eleva la desviación estándar a más de 60,000, comprimiendo a los demás valores en un rango pequeño alrededor de -0.5.

```

1  import pandas as pd
2  from sklearn.preprocessing import StandardScaler
3
4  valores = [15000, 22000, 18000, 28000, 200000, 35000,
5             25000]
6  df = pd.DataFrame(valores, columns=['Valores'])
7
8  std_scaler = StandardScaler()
9  df_escalado = std_scaler.fit_transform(df)
10
11 df_escalado

```

Es importante destacar que ajustamos la clase **StandardScaler** solo una vez (en los datos de entrenamiento) y utilizamos esos parámetros para transformar el conjunto de datos de prueba o cualquier punto de datos nuevo.

2.6.4. Escalado logarítmico

El ajuste de escala logarítmica calcula el logaritmo del valor sin procesar. En teoría, el logaritmo podría tener cualquier base; en la práctica, el ajuste logarítmico suele calcular el logaritmo natural ($\ln$).

$$x_{\ln}^{(i)} = \ln(x) \quad (2.3)$$

donde:

- $x_{\ln}^{(i)}$ es el logaritmo natural de x

El ajuste de escala logarítmica es útil cuando los datos se ajustan a una **distribución de ley de potencias**. En términos generales, una distribución de ley de potencias se ve de la siguiente manera:

- Los valores bajos de X tienen valores muy altos de Y .
- A medida que aumentan los valores de X , los valores de Y disminuyen rápidamente. Por lo tanto, los valores altos de X tienen valores muy bajos de Y .

Las calificaciones de películas son un buen ejemplo de una distribución de ley de potencias. En la siguiente figura, observa lo siguiente:

- La mayoría de las películas tienen muy pocas calificaciones de los usuarios. (Los valores altos de X tienen valores bajos de Y).
- Algunas películas tienen muchas calificaciones de los usuarios. (Los valores bajos de X tienen valores altos de Y).

El ajuste de escala logarítmica cambia la distribución, lo que ayuda a entrenar un modelo que realizará mejores predicciones.

Carlos Santos

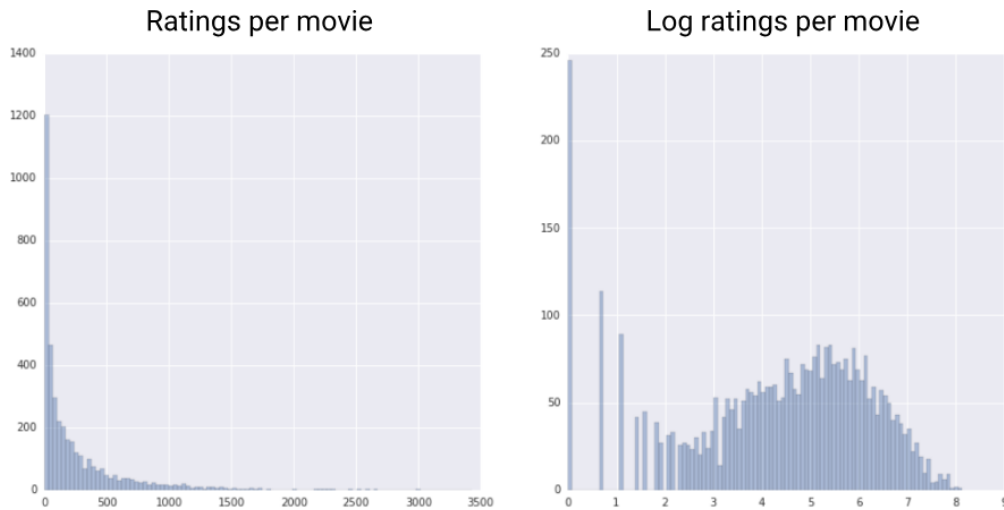


Figura 2.3 – Comparación de una distribución bruta con su logaritmo

Como segundo ejemplo, las ventas de libros se ajustan a una distribución de ley de potencias por los siguientes motivos:

- La mayoría de los libros publicados venden una cantidad muy pequeña de copias, tal vez entre cien y doscientas.
- Algunos libros venden una cantidad moderada de copias, de a miles.
- Solo unos pocos éxitos de ventas venderán más de un millón de copias.

Supongamos que estás entrenando un modelo lineal para encontrar la relación entre, por ejemplo, las portadas de los libros y las ventas de libros. Un modelo lineal que se entrena con valores sin procesar tendría que encontrar algo sobre las portadas de los libros que venden un millón de copias que sea 10,000 veces más potente que las portadas de los libros que venden solo 100 copias. Sin embargo, escalar todos los datos de ventas con un logaritmo hace que la tarea sea mucho más factible. Por ejemplo, el logaritmo de 100 es el siguiente:

$$4,6 = \ln(100)$$

mientras que el logaritmo de 1,000,000 es

$$13,8 = \ln(1,000,000)$$

Por lo tanto, el logaritmo de 1,000,000 es solo tres veces mayor que el logaritmo de 100. Probablemente podrías imaginar que la portada de un libro éxito de ventas es tres veces más potente (de alguna manera) que la portada de un libro que se vende poco.

2.6.5. Recorte

El **recorte** es una técnica para minimizar la influencia de los valores atípicos extremos. En resumen, el recorte suele limitar (reducir) el valor de los valores atípicos a un valor máximo específico. El recorte es una idea extraña y, sin embargo, puede ser muy eficaz.

Por ejemplo, imagina un conjunto de datos que contiene un atributo llamado **roomsPerPerson**, que representa la cantidad de habitaciones (habitaciones totales divididas por la cantidad de ocupantes) de varias casas. En el siguiente gráfico, se muestra que más del 99 % de los valores de los atributos se ajustan a una distribución normal (aproximadamente, una media de 1.8 y una desviación estándar de 0.7). Sin embargo, la función contiene algunos valores atípicos, algunos de ellos extremos:

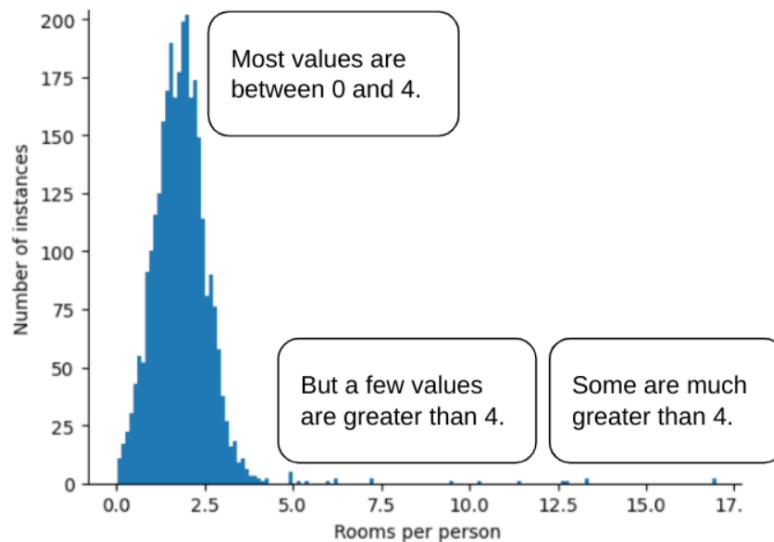


Figura 2.4 – *Principalmente normal, pero no completamente normal.*

¿Cómo puedes minimizar la influencia de esos valores atípicos extremos? Bueno, el histograma no es una distribución uniforme, una distribución normal ni una distribución de ley de potencia. ¿Qué sucede si simplemente limitas o recortas el valor máximo de **roomsPerPerson** en un valor arbitrario, por ejemplo, 4.0?

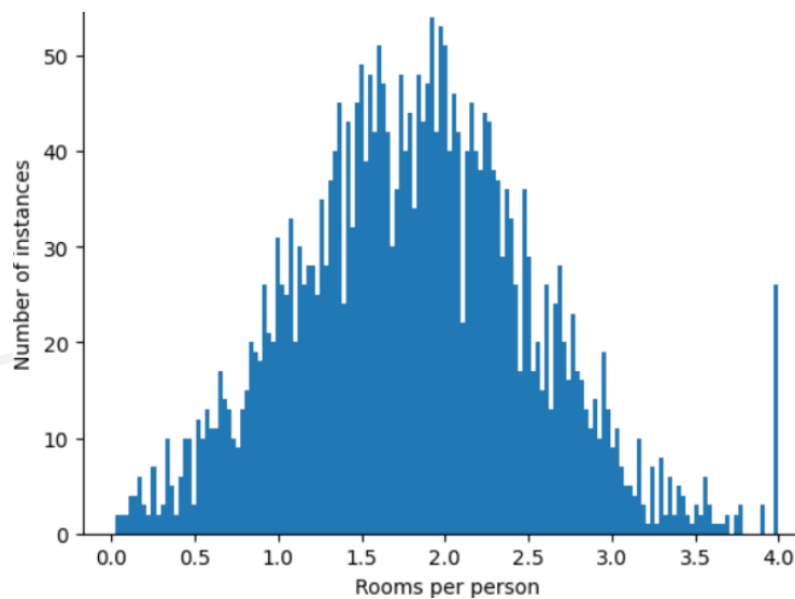


Figura 2.5 – *Valores de la característica de recorte en 4*

Recortar el valor del atributo en 4.0 no significa que tu modelo ignore todos los valores superiores a 4.0. Más bien, significa que todos los valores que eran mayores

que 4.0 ahora se convierten en 4.0. Esto explica la peculiar colina en 4.0. A pesar de esa colina, el conjunto de funciones ajustado ahora es más útil que los datos originales.

¡Espera un segundo! ¿Realmente puedes reducir cada valor atípico a un umbral superior arbitrario? Sí, cuando entrenas un modelo.

También puedes recortar valores después de aplicar otras formas de normalización. Por ejemplo, supongamos que usas el ajuste de escala de la puntuación Z , pero algunos valores atípicos tienen valores absolutos mucho mayores que 3. En este caso, puedes hacer lo siguiente:

- Recorta las puntuaciones Z mayores que 3 para que sean exactamente 3.
- Recorta las puntuaciones Z inferiores a -3 para que sean exactamente -3.

El recorte evita que tu modelo se sobreindexe en datos sin importancia. Sin embargo, algunos valores atípicos son importantes, por lo que debes recortar los valores con cuidado.

2.6.6. Robust Scaler

Otros métodos más avanzados para el escalado de características están disponibles en Scikit - Learn, como **Robust Scaler**, que es especialmente útil y recomendable cuando:

- Estamos trabajando con conjuntos pequeños de datos que contienen muchos valores atípicos.
- Si el algoritmo de aprendizaje automático aplicado a este conjunto de datos es propenso al **sobreajuste**, **RobustScaler** puede ser una buena elección.

El **RobustScaler** busca resistir la influencia de valores atípicos. En lugar de usar la media y la desviación estándar, utiliza la mediana y el rango intercuartílico (IQR).

$$x_{robustscaler}^{(i)} = \frac{x^i - \text{mediana}(x)}{IQR(x)} \quad (2.4)$$

donde el IQR es la diferencia entre el percentil 75 y el percentil 25.

- Operando en cada columna de características de forma independiente, **RobustScaler** elimina el valor mediano y escala el conjunto de datos según el primer y el tercer cuartil del conjunto de datos (es decir, el cuartil 25 y el 75, respectivamente), de forma que los valores más extremos y los valores atípicos se vuelven menos pronunciados.

Definiciones matemáticas

- **Mediana:** es el valor que divide los datos ordenados en dos partes iguales. Matemáticamente, para un conjunto de datos ordenados $x_1 \leq x_2 \leq \dots \leq x_n$, la mediana es

$$\text{mediana}(x) = \begin{cases} x_{\frac{n+1}{2}}, & \text{si } n \text{ es impar} \\ \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}, & \text{si } n \text{ es par} \end{cases}$$

- **Percentil p :** es el valor por debajo del cual cae el p % de los datos. Por ejemplo, el percentil 25 (Q1) deja el 25 % de los datos por debajo, y el percentil 75 (Q3) deja el 75 % por debajo.
- **Rango intercuartílico (IQR):** mide la dispersión de la mitad central de los datos y se calcula como

$$IQR = Q3 - Q1$$

Ejemplo

Consideremos los datos: [15000, 22000, 18000, 28000, 200000, 35000, 25000]

1. Ordenamos los datos:

$$[15000, 18000, 22000, 25000, 28000, 35000, 200000]$$

2. Calculamos la mediana: Hay 7 datos (impar), la mediana es el valor central:

$$\text{mediana} = 25000$$

3. Calculamos los percentiles Q1 y Q3:

- Q1 (percentil 25): mediana de la mitad inferior [15000, 18000, 22000] $\Rightarrow$ 18000
- Q3 (percentil 75): mediana de la mitad superior [28000, 35000, 200000] $\Rightarrow$ 35000

4. Calculamos el IQR:

$$IQR = Q3 - Q1 = 35000 - 18000 = 17000$$

5. Aplicamos la fórmula del RobustScaler a cada dato:

$$x'_i = \frac{x_i - 25000}{17000}$$

$$15000' = \frac{15000 - 25000}{17000} \approx -0,588$$

$$18000' = \frac{18000 - 25000}{17000} \approx -0,412$$

$$22000' = \frac{22000 - 25000}{17000} \approx -0,176$$

$$25000' = \frac{25000 - 25000}{17000} = 0$$

$$28000' = \frac{28000 - 25000}{17000} \approx 0,176$$

$$35000' = \frac{35000 - 25000}{17000} \approx 0,588$$

$$200000' = \frac{200000 - 25000}{17000} \approx 10,294$$

Como se puede ver, los valores normales quedan en un rango aproximado de $[-0.6, 0.6]$, mientras que el valor extremo (200000) aparece muy alejado, pero **no distorsiona la escala de la mayoría de los datos**, que es precisamente la ventaja del **RobustScaler**.

Errores comunes por no escalar correctamente:

1. **Dominio de una variable:** los modelos basados en distancias (como *k-means* o *k-NN*) asignan mayor importancia a variables con grandes magnitudes.
2. **Convergencia lenta o divergente:** en algoritmos de optimización como *Gradient Descent*, una escala desigual provoca gradientes desbalanceados, haciendo que el proceso de entrenamiento sea ineficiente o incluso inestable.
3. **Coeficientes mal interpretados:** en modelos lineales ($y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$), si las variables no están escaladas, la magnitud de β_i no refleja la importancia real de cada atributo.

Ejemplo numérico:

Si aplicamos una estandarización sobre los datos anteriores:

$$x'_1 = \frac{25 - 40}{15} = -1,0, \quad x'_2 = \frac{50,000 - 60,000}{20,000} = -0,5.$$

Ahora ambas variables tienen una escala comparable, permitiendo que el modelo aprenda patrones sin sesgo por magnitud.

En conclusión, el escalado de características no cambia la información que los datos contienen, sino la forma en que los algoritmos la perciben. Es un paso indispensable para garantizar que el aprendizaje se base en la estructura real de los datos y no en su escala numérica.

2.6.7. Conclusiones

El mismo conjunto de datos genera representaciones muy distintas según el método usado:

- **Estandarización:** Útil, pero sensible a valores extremos.
- **Min-Máx:** Sencillo, pero un outlier aplasta el rango de los demás.
- **RobustScaler:** Protege contra outliers y mantiene bien representados los valores centrales.

Este ejemplo muestra que no existe un único ‘mejor’ escalador, sino que la elección depende del problema, de los datos y de los algoritmos que se vaya a utilizar.

Síntesis

Cada método responde a una necesidad distinta:

- La **Estandarización** homogeneiza las unidades estadísticas.

- El **Min-Max** adapta los valores a un rango común.
- El **Robust Scaler** protege contra los valores atípicos.

Comprender estas diferencias permite aplicar el escalado de forma inteligente, escogiendo el método adecuado según el algoritmo, la distribución de los datos y la presencia de valores extremos.

¿Cuándo es necesario escalar?

Si importa el escalado en algoritmos basados en distancias o magnitudes:

- K-Nearest Neighbors (KNN)
- Máquinas de Soporte Vectorial (SVM).
- Regresión logística.
- Redes neuronales.

importa poco o nada en modelos basados en particiones y conteos.

- Árboles de decisión.
- Random Forest.
- Gradient Boosting (XGBoost, LighGBM)

¿Qué columnas deben escalarse?

Non todo lo que está en una tabla de datos necesita pasar por un escalador. Las variables **numérica continuas** son las candidatas naturales: Edad, ingreso, temperatura, altura, peso. Allí el escalado ayuda a evitar que una magnitud domine a otra solo por la escala de sus números.

En cambio, las varianles **categorías codificadas en dummies** (como hombre/-mujer, tiene auto / no tiene auto), **NO deben escalarse**, porque perderían su significado binario. Tampoco se escalan identificadores o claves como ID del cliente.º “número de serie” , ya que no representan magnitudes comparables.

Dicho de otro modo, antes de aplicar cualquier técnica conviene **clasificar las columnas** de acuerdo con su naturaleza. Escalar solo tiene sentido en aquellas que expresan cantidades medibles y continuas.

¿Se escalan todas de la misma manera?

Una vez decidido qué variables son candidatas al escalado, surge otra cuestión: ¿todas deben transformarse con el mismo método? La respuesta es no. **Cada variable puede necesitar un tratamiento distinto según sus características.**

- Una distribución cercana a la normal se ajusta bien con la estandarización.
- Una variable fuertemente sesgada por valores extremos se beneficia más de un escalado robusto.

- Una variable naturalmente acotada, como una calificación de 0 a 100 o un porcentaje, suele representarse mejor con Min-Max.

Así, el escalado no es un ritual uniforme, sino un proceso adaptado: se elige la herramienta apropiada para cada tipo de dato, como un mecánico que usa una llave distinta según el tornillo que debe ajustar.

El problema de la fuga de datos

En este punto aparece una precaución crucial. Uno de los errores más comunes es aplicar el escalado antes de separar el conjunto en entrenamiento y prueba. Si calculamos la media y la desviación estándar con todos los datos, el conjunto de prueba ya no es realmente “desconocido” para el modelo: parte de su información se filtró en el proceso de preparación. Esto se llama **fuga de datos**.

La fuga de datos genera la ilusión de un modelo más preciso de lo que en verdad es, porque al entrenar ya tuvo acceso indirecto a estadísticas del conjunto de prueba. Para evitarla, el procedimiento correcto es siempre el mismo: primero dividir en entrenamiento y prueba; después ajustar el escalador únicamente con los datos de entrenamiento; y por último aplicar esa misma transformación tanto al entrenamiento como a la prueba.

La analogía es la de un examen. El cuestionario de práctica (entrenamiento) se diseña y resuelve antes. El examen real (prueba) debe mantenerse oculto hasta el momento de la evaluación. Si el profesor usara información del examen en la preparación, estaría dando una ventaja injusta y el resultado final perdería validez. De la misma manera, en el escalado, los parámetros deben calcularse solo con los datos de entrenamiento y luego aplicarse con rigor a los de prueba.

Conclusión general

El escalado de variables no es un adorno matemático: es un paso de higiene y de justicia numérica en la construcción de modelos. Requiere decidir qué columnas se escalan, elegir el método apropiado para cada una y aplicar el procedimiento de manera correcta para no incurrir en fuga de datos. Solo así garantizamos que las comparaciones entre variables sean legítimas y que la evaluación final del modelo refleje su verdadera capacidad de generalizar a datos nuevos.

2.6.8. Aplicación en Python

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler,
   MinMaxScaler, RobustScaler
4 from sklearn.compose import ColumnTransformer
5 datos = pd.read_csv("data/datos_para_escalado.csv")
6 num_cols = datos.select_dtypes(include=[np.number]).columns
   # Selecciona las columnas numericas del DataFrame 'datos'
7
8 cols_con_outliers = [] # Lista para guardar los nombres de
   las columnas que contienen outliers
9
```

```

10
11
12 for c in num_cols: # Itera sobre cada columna numerica
13     s = pd.to_numeric(datos[c], errors="coerce").dropna()
14     # Convierte la columna a numerica y elimina
15     # valores NaN
16     if s.empty: # Si la serie esta vacia, pasa a la
17         # siguiente columna
18         continue
19     q1, q3 = s.quantile(0.25), s.quantile(0.75) #
20     # Calcula el primer y tercer cuartil
21     iqr = q3 - q1 # Calcula el rango intercuartilico (
22     # IQR)
23     if iqr == 0: # Si el IQR es 0, no hay dispersion (
24     # todos los valores iguales), se salta
25     continue
26     lim_inf, lim_sup = q1 - 1.5*iqr, q3 + 1.5*iqr #
27     # Calcula los limites inferior y superior para
28     # detectar outliers
29     n_out = int(((s < lim_inf) | (s > lim_sup)).sum()) #
30     # Cuenta cuantos valores estan fuera de esos
31     # limites
32     if n_out > 0: # Si hay al menos un outlier...
33         cols_con_outliers.append(c) # ...agrega el
34         # nombre de la columna a la lista
35
36 cols_con_outliers # Muestra la lista final de columnas que
37 # contienen outliers
38
39 # Definir columnas por tipo de escalado
40 cols_std = ["edad", "presion_sistolica"]
41 cols_minmax = ["porcentaje_asistencia", "calificacion_examen"
42               , "horas_estudio_diarias"]
43 cols_robust = ["gasto_medico_anual"]
44
45 # Construir el transformador por columnas
46
47 '''
48 Usamos ColumnTransformer para aplicar **distintas
49 transformaciones a distintos subconjuntos
50 de columnas en una sola pasada. Cada tupla dentro de
51 transformers=[...]
52 tiene la forma (nombre_bloque, transformador,
53                 columnas_objetivo).
54
55 - ("std", StandardScaler(), cols_std)
56 Aplica estandarizacion a las columnas listadas en cols_std
57 - ("minmax", MinMaxScaler(), cols_minmax)
58 Aplica normalizacion MinMax a las columnas de cols_minmax
59 - ("robust", RobustScaler(quantile_range=(25, 75)),
60   cols_robust)

```

```

44 Aplica escalado robusto a las columnas de cols_robust
45
46 Parametros adicionales del 'ColumnTransformer':
47
48 - remainder="drop"
49 Indica que las columnas no listadas en los bloques anteriores
   se descartan de la salida.
50 Alternativa: "passthrough" para dejarlas pasar sin
   transformar)
51
52 - verbose_feature_names_out=False
53 Mantiene nombres de salida 'limpios' (p. ej., 'edad') en
   lugar de prefijarlos con el nombre
54 del bloque (p. ej., 'std_edad').
55 '''
56
57 preprocesador = ColumnTransformer(
58 transformers=[
59 ("std", StandardScaler(), cols_std),
60 ("minmax", MinMaxScaler(), cols_minmax),
61 ("robust", RobustScaler(quantile_range=(25, 75)), cols_robust
   ),
62 ],
63 remainder="drop",
64 verbose_feature_names_out=False
65 )
66
67 # Ajustar (fit) sobre TODO el dataset
68 preprocesador.fit(datos)
69
70 # Transformar con los parametros aprendidos
71 X_escalado = preprocesador.transform(datos)
72
73 # Reconstruir DataFrame con los mismos nombres de columnas
74
75 columnas_escaladas = preprocesador.get_feature_names_out()
76 datos_escalado = pd.DataFrame(X_escalado, columns=
   columnas_escaladas, index=datos.index)
77 datos_escalado
78
79 # --- Reordenar para que siga el orden original del CSV ---
80 orden_original = [c for c in datos.columns if c in
   datos_escalado.columns]
81 datos_escalado = datos_escalado[orden_original]
82 datos_escalado
83 datos_escalado.to_csv("datos_ya_escalados.csv")

```

2.7 Escalado de variables Categoricals

A menudo, conviene representar las características que contienen valores enteros como **datos categóricos** en lugar de numéricos. Por ejemplo, si consideramos una

característica de código postal cuyos valores son enteros. Si representamos esta característica numéricamente en lugar de **categoricamente**, le estamos pidiendo al modelo que encuentre una relación numérica entre los diferentes códigos postales. Es decir, le estamos indicando al modelo que trate el código postal 200004 como una señal dos veces (o la mitad) de grande que el código postal 100002. Representar los códigos postales como datos categóricos permite al modelo ponderar cada código postal individualmente

La **codificación** consiste en convertir datos categóricos u otros datos en vectores numéricos con los que un modelo pueda entrenarse. Esta conversión es necesaria porque los modelos solo pueden entrenarse con valores de **punto flotante**; no pueden entrenarse con cadenas.

Cuando hablamos de datos categóricos, tenemos que distinguir entre características **ordinales** y **nominales**. Las características ordinales pueden entenderse como valores categóricos que pueden clasificarse u ordenarse. Por ejemplo, la talla de las camisetas sería un rasgo ordinal, porque podemos definir un orden: $XL > L > M$. En cambio, las características nominales no implican un orden; por seguir con el ejemplo anterior, podríamos pensar que el color de la camiseta es un rasgo nominal, ya que normalmente no tiene sentido decir que, por ejemplo, el rojo es mayor que el azul.

- Una tentación podría ser reemplazar las categorías con números arbitrarios (ejemplo: rojo = 1, azul = 2, verde = 3). Pero esto introduce un error: el modelo interpretará que **existe un orden o una distancia entre las categorías** que en realidad no tiene sentido (¿acaso “azul” es mayor que “rojo”?).
- Un teatro tiene localidades: Orquesta, Platea, Balcón. Si las numeramos como 1, 2 y 3, parecería que hay una jerarquía: Balcón > Platea > Orquesta. En realidad solo son etiquetas. La codificación correcta sería dar a cada sección su propio boleto sin relación de magnitud entre ellas.

2.7.1. Codificación Nominal (One-Hot)

La idea que hay detrás de este enfoque es crear una nueva característica ficticia (también llamada *dummy variable*) para cada valor único en una columna categórica nominal. De esta manera, cada categoría se representa mediante un vector binario que indica la presencia o ausencia de dicha categoría.

Para realizar esta transformación, podemos usar el objeto **OneHotEncoder**, implementado en el módulo **preprocessing** de **Scikit-Learn**.

En una codificación **one-hot**:

- Cada categoría se representa mediante un vector (array) de N elementos donde N es el número de categorías. Por ejemplo, si **Color** hay 3 categorías posibles, el vector **one-hot** que las representa tendrá 3 elementos.
- Exactamente uno de los elementos de un vector **one-hot** tiene el valor 1; todos los elementos restantes tienen el valor 0

Ejemplo: Supongamos que tenemos una variable categórica **Color** con los valores {Rojo, Verde, Azul}. La codificación *One-Hot* produce las siguientes variables binarias:

Color	Rojo	Verde	Azul
Rojo	1	0	0
Verde	0	1	0
Azul	0	0	1

Implementación básica en Python:

```

1 from sklearn.preprocessing import OneHotEncoder
2 import pandas as pd
3
4 # Datos de ejemplo
5 datos = pd.DataFrame({'Color': ['Rojo', 'Verde', 'Azul', '
    Rojo']})
6
7 # Inicializamos el codificador
8 encoder = OneHotEncoder(sparse_output=False)
9
10 # Ajustamos y transformamos
11 codificado = encoder.fit_transform(datos[['Color']])
12
13 # Mostramos el resultado
14 print(pd.DataFrame(codificado, columns=encoder.
    get_feature_names_out(['Color'])))

```

Uso de ColumnTransformer: Cuando un conjunto de datos tiene tanto variables numéricas como categóricas, es común aplicar diferentes transformaciones a cada tipo de variable. Para ello, se utiliza la clase **ColumnTransformer**, que permite aplicar transformaciones específicas a columnas concretas.

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.preprocessing import OneHotEncoder,
    StandardScaler
3
4 # Supongamos que tenemos columnas categoricas y numericas
5 columnas_categoricas = ['Color']
6 columnas_numericas = ['Precio', 'Peso']
7
8 # Definimos las transformaciones
9 transformador = ColumnTransformer(
10     transformers=[
11         ('cat', OneHotEncoder(sparse_output=False, drop='first'),
            columnas_categoricas),
12         ('num', StandardScaler(), columnas_numericas)
13     ],
14     remainder='passthrough' # Mantiene las columnas no
        transformadas
15 )

```

Parámetro `remainder='passthrough'`: El argumento `remainder` indica qué hacer con las columnas que no se incluyen explícitamente en los transformadores.

- `remainder='drop'` (valor por defecto): elimina las columnas no especificadas.
- `remainder='passthrough'`: deja pasar esas columnas sin modificarlas.

Esto resulta útil cuando solo queremos transformar algunas variables (por ejemplo, las categóricas) y mantener las numéricas tal como están.

Parámetro `handle_unknown='ignore'`: Este parametro es fundamental cuando el modelo se entrena con un conjunto de datos que no contiene todas las categorias posibles. Si durante la prediccion aparece una categoria nueva (no vista durante el fit), el encoder la ignorara en lugar de generar un error.

- Si `handle_unknown='error'` (por defecto), el encoder lanzara un error al encontrar una categoria desconocida.
- Si `handle_unknown='ignore'`, las categorias desconocidas se codifican como un vector de ceros.

Ejemplo:

```
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

Esto es muy util en produccion, donde pueden aparecer valores nuevos que no estaban presentes durante el entrenamiento.

Eliminación de una columna: `drop='first'`

```
1 OneHotEncoder(drop='first')
```

Cuando utilizamos conjuntos de datos codificados en **one-hot**, tenemos que tener en cuenta que este hecho introduce **multicolinealidad**, que puede ser un problema para ciertos métodos (por ejemplo, los métodos que requieren la inversión de matrices). Si las características están muy correlacionadas, es difícil realizar la inversión de las matrices desde el punto de vista de la computación, lo que puede dar lugar a estimaciones numéricamente inestables.

Para reducir la **correlación** entre las variables, podemos eliminar una columna de características del array codificado en **one-hot**. Tenga en cuenta que no perdemos ninguna información importante al eliminar una columna de características.

Por ejemplo:

Color	Verde	Azul
Rojos	0	0
Verde	1	0
Azul	0	1

Si eliminamos la columna "Rojo", la información de característica se sigue conservando, ya que si observamos 'Verde'=0 y 'Azul'=0, implica que la observación debe ser "Rojo".

Ventajas:

- No introduce orden artificial entre categorías.
- Compatible con la mayoría de los modelos de Machine Learning.

Desventajas:

- Aumenta la dimensionalidad del conjunto de datos cuando hay muchas categorías.
- Puede generar redundancia si no se elimina una columna (multicolinealidad).

Valores atípicos en datos categóricos

Al igual que los datos numéricos, los datos categóricos también contienen valores atípicos. Supongamos que un **Color** conjunto de datos contienen no solo los colores más comunes, sino también algunos colores atípicos poco frecuentes, como el gris rata o el café madera. En lugar de asignar una categoría separada a cada uno de estos colores atípicos, se pueden agrupar en una única categoría general denominada "Fuera de vocabulario" (OOV). En otras palabras, todos los colores atípicos se agrupan en una sola categoría. El sistema aprende un único peso para esta categoría.

2.7.2. Cruces de características

Las **combinaciones de atributos** se crean combinando (tomando el producto cartesiano) de dos o más características categóricas o agrupadas del conjunto de datos. Al igual que las transformaciones de polinomios, las combinaciones de atributos permiten que los modelos lineales manejen las no linealidades. Las combinaciones de atributos también codifican las interacciones entre las características

Por ejemplo, considera un conjunto de datos de hoja con los atributos categóricos:

- **edges**, que contiene los valores smooth, toothed y lobed
- **arrangement**, que contiene los valores opposite y alternate

Supongamos que el orden anterior es el orden de las columnas de atributos en una representación one-hot, de modo que una hoja con **smooth** y **opposite** se representa como $\{(1, 0, 0), (1, 0)\}$.

La combinación de atributos, o producto cartesiano, de estos dos atributos sería la siguiente:

$$\left\{ \begin{array}{l} \text{Smooth_Opposite, Smooth_Alternate, Toothed_Opposite,} \\ \text{Toothed_Alternate, Lobed_Opposite, Lobed_Alternate} \end{array} \right\}$$

en el que el valor de cada término es el producto de los valores de los atributos base, de modo que:

$\text{Smooth_Opposite} = \text{edges}[0] \cdot \text{arrangement}[0],$
 $\text{Smooth_Alternate} = \text{edges}[0] \cdot \text{arrangement}[1],$
 $\text{Toothed_Opposite} = \text{edges}[1] \cdot \text{arrangement}[0],$
 $\text{Toothed_Alternate} = \text{edges}[1] \cdot \text{arrangement}[1],$
 $\text{Lobed_Opposite} = \text{edges}[2] \cdot \text{arrangement}[0],$
 $\text{Lobed_Alternate} = \text{edges}[2] \cdot \text{arrangement}[1]$

Por ejemplo, si una hoja tiene un `edges=lobed` y una `arrangement = alternate`, el vector de combinación de características tendrá un valor de 1 para `Lobed_Alternate` y un valor de 0 para todos los demás términos:

`{0, 0, 0, 0, 0, 1}`

Este conjunto de datos se podría usar para clasificar las hojas por especie de árbol, ya que estas características no varían dentro de una especie.

Las combinaciones de características son, en cierto modo, análogas a las **transformaciones polinómicas**. Ambas combinan múltiples características en una nueva característica sintética que el modelo puede usar para entrenarse y aprender no linealidades. Las transformaciones polinómicas suelen combinar datos numéricos, mientras que las combinaciones de características combinan datos categóricos.

2.7.3. Codificación Ordinal

La codificación **Ordinal** consiste en asignar un número entero a cada categoría dentro de una variable categórica. A diferencia de la codificación *One-Hot*, en este caso las categorías se representan con valores numéricos que pueden reflejar o no un orden natural entre ellas.

En **Scikit-Learn**, esta transformación se implementa mediante la clase **OrdinalEncoder**, del módulo **preprocessing**.

Ejemplo: Supongamos que tenemos una variable categórica **Tamaño** con los valores `{Pequeno, Mediano, Grande}`. La codificación ordinal puede representarlas así:

Tamaño	Valor codificado
Pequeno	0
Mediano	1
Grande	2

Implementación básica en Python:

```

1 from sklearn.preprocessing import OrdinalEncoder
2 import pandas as pd
3
4 # Datos de ejemplo
5 datos = pd.DataFrame({'Tamaño': ['Pequeno', 'Mediano', 'Grande', 'Mediano']})
6
7 # Inicializamos el codificador
8 encoder = OrdinalEncoder(categories=[['Pequeno', 'Mediano', 'Grande']])

```



```

9
10 # Ajustamos y transformamos
11 codificado = encoder.fit_transform(datos[['Tamano']])
12
13 # Mostramos el resultado
14 print(pd.DataFrame(codificado, columns=['Tamano_codificado']))

```

Importancia del orden: Este metodo es especialmente adecuado cuando las categorias tienen un **orden logico o jerarquico** (por ejemplo: bajo < medio < alto, o pequeno < mediano < grande). Sin embargo, si las categorias no tienen un orden natural (por ejemplo, Color: rojo, verde, azul), esta codificacion podria inducir al modelo a interpretar relaciones inexistentes.

Uso de ColumnTransformer: En conjuntos de datos mixtos, podemos aplicar la codificacion ordinal solo a ciertas columnas, combinandola con otras transformaciones para columnas numericas o nominales.

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.preprocessing import OrdinalEncoder,
   StandardScaler
3
4 # Definimos las columnas
5 columnas_ordinales = ['Tamano']
6 columnas_numericas = ['Precio']
7
8 # Creamos el transformador
9 transformador = ColumnTransformer(
10 transformers=[
11 ('ord', OrdinalEncoder(categories=[['Pequeno', 'Mediano', '
   Grande']]), columnas_ordinales),
12 ('num', StandardScaler(), columnas_numericas)
13 ],
14 remainder='passthrough' # Deja pasar las demas columnas sin
   modificar
15 )

```

Manejo de categorias desconocidas: `handle_unknown="use_encoded_value"` En algunos casos, durante la fase de prediccion, pueden aparecer categorias que no estaban presentes en los datos de entrenamiento. Por defecto, `OrdinalEncoder` genera un error cuando encuentra una categoria desconocida. Para evitar este problema, se puede usar el parametro:

```

1 handle_unknown="use_encoded_value", unknown_value=-1

```

Significado:

- `handle_unknown="use_encoded_value"` indica al codificador que, en lugar de lanzar un error, asigne un valor numerico especifico a las categorias no vistas.

- `unknown_value=-1` define cual sera ese valor numerico asignado a las categorias desconocidas.

Ejemplo:

```

1  from sklearn.preprocessing import OrdinalEncoder
2  import pandas as pd
3
4  # Datos de entrenamiento
5  train = pd.DataFrame({'Tamano': ['Pequeno', 'Mediano',
6  , 'Grande']})
7
8  # Datos nuevos con una categoria no vista ("
9  ExtraGrande")
10 test = pd.DataFrame({'Tamano': ['Mediano', '
11 ExtraGrande']})
12
13 # Codificador con manejo de categorias desconocidas
14 encoder = OrdinalEncoder(
15     categories=[['Pequeno', 'Mediano', 'Grande']],
16     handle_unknown='use_encoded_value',
17     unknown_value=-1
18 )
19
20 # Entrenamos y transformamos
21 encoder.fit(train[['Tamano']])
22 print(encoder.transform(test[['Tamano']]))

```

Salida esperada:

Tamano	Codificado
Mediano	1
ExtraGrande	-1

Interpretacion: En este ejemplo, la categoria `ExtraGrande` no se encontraba en los datos de entrenamiento. Gracias a los parametros `handle_unknown='use_encoded_value'` y `unknown_value=-1`, el encoder le asigna el valor `-1` en lugar de generar un error.

Ventajas:

- Evita errores cuando aparecen nuevas categorias en datos futuros.
- Permite conservar la compatibilidad del modelo en produccion.

Desventajas:

- El valor asignado (`-1`) no representa un orden real, por lo que debe manejarse con cuidado.
- Algunos modelos pueden interpretar el valor `-1` como una posicion inferior en el orden, lo cual puede sesgar los resultados.

Recomendacion: Usar esta configuracion cuando:

- Se espera que en produccion puedan aparecer categorias nuevas.
- El modelo utilizado no se vea afectado por el valor numerico asignado (por ejemplo, arboles de decision o *gradient boosting*).

Parametro remainder='passthrough': Al igual que con `OneHotEncoder`, este parametro permite mantener las columnas no transformadas. Esto es util cuando solo algunas variables requieren codificacion.

Cuando usar la codificacion ordinal:

- Cuando las categorias tienen un orden natural (por ejemplo: `educacion`: primaria, secundaria, universitaria).
- Cuando el modelo puede interpretar correctamente la relacion entre los valores (como arboles de decision, *gradient boosting*, etc.).

Cuando no usarla:

- Si las categorias no tienen relacion de orden (por ejemplo, nombres de ciudades o colores).
- En modelos lineales o basados en distancia (como *k-NN* o *SVM*), donde la magnitud de los numeros puede inducir interpretaciones erroneas.

Ventajas:

- Simple y eficiente (solo agrega una columna por variable).
- No incrementa la dimensionalidad del conjunto de datos.

Desventajas:

- Introduce una relacion ordinal incluso si no existe.
- Puede sesgar modelos sensibles a las magnitudes numericas.

2.8 Valores faltantes

Además de comprender por qué aparecen los valores faltantes, es importante reconocer que su tratamiento puede modificar significativamente el desempeño del modelo. Imputar no es solo “rellenar huecos”, sino tomar una decisión estadística que cambia la distribución de los datos y, en consecuencia, el aprendizaje del algoritmo.

Una de las formas más sencillas de tratar los datos que faltan es, simplemente, eliminar completamente las correspondientes características (columnas) o los ejemplos de entrenamiento (filas) del conjunto de datos;

1. Las filas con valores que faltan pueden eliminarse fácilmente mediante el método **dropna**.
2. De la misma manera, podemos eliminar las columnas que tienen al menos un NaN en cualquier fila estableciendo el argumento de **axis=1**. (A menudo, la eliminación de columnas enteras de características no es factible, ya que podríamos perder demasiados datos valiosos.)
3. Podemos sustituir los valores faltantes por estadísticas descriptivas (media, mediana, moda) o un valor constante.

```

1      from pathlib import Path
2      import pandas as pd
3      import tarfile
4      import urllib.request
5      import warnings
6      warnings.filterwarnings("ignore")
7
8
9      def load_housing_data():
10         tarball_path = Path("datasets/housing.tgz")
11         if not tarball_path.is_file():
12             Path("datasets").mkdir(parents=True, exist_ok=True)
13             url = "https://github.com/ageron/data/raw/main/
14                  housing.tgz"
15             urllib.request.urlretrieve(url, tarball_path)
16             with tarfile.open(tarball_path) as housing_tarball:
17                 housing_tarball.extractall(path="datasets")
18             return pd.read_csv(Path("datasets/housing/housing.csv"))
19
20         housing = load_housing_data()
21         housing.info() #Con esto podemos tener una
22                       #visualizacion previa de las columnas del DF
23         #option 1
24         #housing.dropna(subset=["total_bedrooms"], inplace=
25                       #True) #Elimina las filas donde hay nulos en la
26                       #columna 'total_bedrooms'
27         # option 2
28         #housing.drop("total_bedrooms", axis=1)      # Elimina
29                       #la columna completa total_bedrooms.
30         # option 3
31         #median = housing["total_bedrooms"].median()
32         #housing["total_bedrooms"].fillna(median, inplace=
33                       #True) #Calcula la mediana de la columna
34                       #total_bedrooms. Rellena los valores faltantes (NaN
35                       #) con esa mediana.

```

Tipos de valores faltantes

No todos los huecos son iguales. En ciencia de datos distinguimos tres mecanismos:

- **MCAR (Missing Completely At Random):** el valor falta totalmente al azar. No depende ni de la variable faltante ni de otras variables del dataset. Es el caso menos problemático.
- **MAR (Missing At Random):** el valor falta de forma relacionada con otras variables, pero no con la variable faltante en sí. Por ejemplo, personas jóvenes tienden a omitir su salario en una encuesta.
- **MNAR (Missing Not At Random):** el valor falta precisamente por el valor que no se observa. Por ejemplo, personas con salarios muy altos podrían deliberadamente no reportarlo.

Identificar el mecanismo de ausencia es clave, porque determina qué técnicas de imputación serán apropiadas. Métodos simples como la media o la mediana funcionan bien bajo MCAR o MAR, pero pueden introducir sesgos importantes bajo MNAR.

Consecuencias de una imputación ingenua

Reemplazar los huecos sin analizar puede traer efectos secundarios no deseados:

- **Reducción de la varianza:** imputar con la media aplasta las diferencias entre observaciones.
- **Distorsión de correlaciones:** valores inventados pueden alterar la relación entre variables.
- **Creación de patrones ficticios:** imputaciones mal elegidas pueden inducir tendencias inexistentes.

Por eso, incluso métodos “simples” requieren reflexión.

Estrategias comunes de imputación

En la práctica, existen varias familias de métodos:

- **Imputación Simple (SimpleImputer):** Sustituye valores faltantes por estadísticas descriptivas (media, mediana, moda) o un valor constante. Es rápida y estable, pero puede ser demasiado simplista.
- **Imputación por Vecinos (KNNImputer):** Cada valor faltante se reemplaza usando observaciones similares. Captura mejor la estructura del dataset, pero puede ser costoso en datos grandes.
- **Imputación Iterativa (IterativeImputer / MICE):** Modela cada característica con valores faltantes en función de las demás y repite el proceso de forma iterativa. Es más potente y flexible, especialmente cuando las relaciones entre variables son complejas.
- **Indicadores de ausencia:** Agregar una columna binaria que marque qué valores fueron imputados. Esto permite al modelo distinguir entre datos reales y estimados.

Buenas prácticas

- **Explorar antes de imputar:** visualizar patrones de ausencia, distribuciones y correlaciones.
- **Imputar dentro de un Pipeline:** asegura que la misma estrategia se aplique consistentemente y evita errores humanos.
- **Comparar métodos:** no asumir que la media o KNN serán óptimos; validar mediante experimentos.
- **Evitar borrar filas:** eliminar datos sólo si la cantidad de huecos es pequeña o la columna es irrelevante.

En resumen, tratar valores faltantes es una etapa crítica del preprocesamiento. La calidad de la imputación puede determinar si un modelo fracasa o alcanza su mejor desempeño. Aplicar las técnicas adecuadas y evitar la fuga de datos asegura que el modelo aprenda de manera honesta y generalice correctamente.

Fuga de datos

Al igual que en la codificación de variables categóricas, aquí también debemos cuidar la fuga de datos. **Nunca se deben calcular promedios o medianas usando todo el dataset** antes de separar en entrenamiento y prueba. Si lo hacemos, el modelo ya estaría “viendo” información del conjunto de prueba durante el entrenamiento. El paralelo en el examen sería que el niño recibe de antemano las respuestas del examen final disfrazadas en el cuestionario de práctica. El resultado será una calificación irrealmente alta. Por eso, la regla de oro es:

1. Dividir primero en entrenamiento y prueba.
2. Ajustar (‘fit’) el imputador solo con el entrenamiento.
3. Aplicar (‘transform’) esa misma estrategia en entrenamiento y prueba.

2.8.1. Simple Imputer

`SimpleImputer` es el método más básico y directo para reemplazar valores faltantes (NaN) en un conjunto de datos. Forma parte del módulo `sklearn.impute` y permite utilizar reglas simples y determinísticas para rellenar huecos antes de entrenar un modelo de aprendizaje automático.

¿Por qué usar SimpleImputer?

La mayoría de los algoritmos de Machine Learning no admiten datos faltantes. `SimpleImputer` nos permite:

- Estandarizar una estrategia coherente de imputación.
- Integrar el proceso dentro de un **Pipeline**.
- Evitar la fuga de datos al ajustar solo con el conjunto de entrenamiento.

- Proveer un punto de partida rápido para experimentación.

Aunque es una técnica sencilla, **suele ser eficaz cuando la proporción de valores faltantes es baja o cuando no existen patrones complejos en los datos.**

Estrategias disponibles

El parámetro `strategy` define cómo se rellenarán los valores faltantes:

- **mean**: reemplaza con la media de la columna (solo numérica).
- **median**: usa la mediana (más robusto ante valores extremos).
- **most_frequent**: utiliza el valor más repetido. Útil para categorías.
- **constant**: rellena usando el valor especificado en `fill_value`.

Ejemplo básico de uso

```

1  from sklearn.impute import SimpleImputer
2  import numpy as np
3
4  data = np.array([
5      [1, 2],
6      [np.nan, 3],
7      [7, np.nan]
8  ])
9
10 imputer = SimpleImputer(strategy='mean')
11 X_filled = imputer.fit_transform(data)

```

- **fit()** Calcula la estadística (media, mediana, etc.).
- **transform()** Aplica esa estrategia para reemplazar los NaN.

Parámetros importantes

- **strategy**: tipo de imputación (media, mediana, moda o constante).
 - **mean**: Rellena con la *media* de la columna. *Ejemplo*: si la columna es [2, 4, 6, NaN], el valor imputado será 4.
 - **median**: Usa la *mediana*. *Ejemplo*: [1, 10, 3, NaN] → mediana = 3.
 - **most_frequent**: Utiliza el *valor más frecuente* (moda). *Ejemplo*: [7, 7, 9, NaN] → imputación: 7.
Para variables categóricas: ['rojo', 'azul', 'rojo', NaN] → imputación: rojo.
 - **constant**: Reemplaza con un `fill_value` definido por el usuario. *Ejemplos comunes*:
 - Para números: 0, -1, 999, 0.0

- Para categorías: "missing", "desconocido", "NA"
- **fill_value**: valor específico cuando **strategy='constant'**.
- **missing_values**: símbolo que representa valores faltantes; por defecto es `np.nan`.
- **add_indicator**: si es `True`, agrega columnas binarias que indican qué valores fueron imputados.

Imputación por columna

Cada columna se trata de manera independiente. Por ejemplo, si se imputa con **strategy='mean'**:

$$\text{columna}_j^{\text{imputada}} = \begin{cases} x_{ij} & \text{si } x_{ij} \text{ no es NaN} \\ \bar{x}_j & \text{si } x_{ij} \text{ es NaN} \end{cases}$$

```

1      import pandas as pd
2      from io import StringIO
3      import sys
4      from sklearn.impute import SimpleImputer
5      import numpy as np
6
7
8      csv_data = \
9          '''A,B,C,D
10         1.0,2.0,3.0,4.0
11         5.0,6.0,,8.0
12         10.0,11.0,12.0,'''
13
14         # If you are using Python 2.7, you need
15         # to convert the string to unicode:
16
17         if (sys.version_info < (3, 0)):
18             csv_data = unicode(csv_data)
19
20         df = pd.read_csv(StringIO(csv_data))
21         # impute missing values via the column mean
22
23
24
25         imr = SimpleImputer(missing_values=np.nan, strategy='
                mean', add_indicator=True)
26         imr = imr.fit(df.values)
27         imputed_data = imr.transform(df.values)
28         imputed_data

```

Ventajas

- La ventaja es que almacenará el valor (mean, median, most recuente, constant) de cada característica, lo que permitirá imputar valores faltantes no solo con

el conjunto de entrenamiento, sino también con el conjunto de validación, el conjunto de prueba y cualquier dato nuevo que se introduzca en el modelo.

- Muy rápido y fácil de aplicar.
- No requiere hiperparámetros complejos.
- Proporciona un punto de referencia para métodos más avanzados.
- Puede combinarse de forma segura con modelos dentro de un Pipeline.

Desventajas

- No captura relaciones entre variables.
- Reduce la varianza al reemplazar huecos con valores centrales.
- Puede introducir sesgos si los datos faltan por mecanismos no aleatorios (MNAR).

2.8.2. Uso dentro de un Pipeline

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LogisticRegression
4
5 pipeline = Pipeline(steps=[
6     ('imputer', SimpleImputer(strategy='median')),
7     ('scaler', StandardScaler()),
8     ('model', LogisticRegression())
9 ])
10
11 pipeline.fit(X_train, y_train)
```

Esto garantiza que la imputación ocurre sólo con los datos de entrenamiento y se aplica de manera idéntica al conjunto de prueba.

Cuándo usar SimpleImputer

Úsalo cuando:

- La proporción de valores faltantes es baja o moderada.
- No existen patrones complejos de ausencia.
- Necesitas un método base para comparación.
- Trabajas con modelos sensibles a la presencia de huecos (SVM, regresión logística, redes neuronales, etc.).

En caso de relaciones más ricas entre variables, métodos como `KNNImputer` o `IterativeImputer` pueden ser más adecuados.

- No se puede garantizar que no haya valores faltantes en los nuevos datos una vez que el sistema esté en funcionamiento, por lo que es más seguro aplicarlo **imputer** a todos los atributos numéricos.

2.9 Análisis Exploratorio de Datos (EDA) para Machine Learning

Después de recopilar y limpiar los datos, la siguiente etapa fundamental es el **Análisis Exploratorio de Datos (EDA)**, que permite comprender mejor las características de los datos antes de aplicar cualquier modelo de Machine Learning. La información obtenida durante el EDA influye directamente en las decisiones de todo el proyecto.

Objetivos del EDA

- Comprender la **distribución de los datos**.
- Identificar patrones, tendencias y anomalías.
- Guiar decisiones de preprocesamiento, como:
 - Detección y manejo de **valores atípicos**.
 - Transformación o **escalado de características**.
 - Selección de **features**.
 - Elección del **algoritmo de Machine Learning**.

Distribuciones y métricas cuantitativas

Aunque las visualizaciones (como histogramas y boxplots) ayudan a entender la distribución de los datos, es necesario usar métricas confiables para cuantificar sus características. Dos métricas clave son:

1. Asimetría (Skewness):

- Mide el grado de **simetría de la distribución**.
- Permite comparar la distribución con una **distribución normal perfecta**.
- Tipos de asimetría:
 - Asimetría positiva: cola derecha más larga.
 - Asimetría negativa: cola izquierda más larga.

2. Curtosis (Kurtosis):

- Indica la **concentración de datos en los extremos** de la distribución.
- Ayuda a identificar colas pesadas o ligeras comparadas con la normal.
- Tipos de curtosis:
 - Mesocúrtica: similar a la normal.
 - Leptocúrtica: colas más pesadas y pico más alto.
 - Platicúrtica: colas más ligeras y pico más bajo.

Importancia para Machine Learning

- La asimetría y la curtosis afectan la **selección de algoritmos** y la **precisión del modelo**.
- Transformaciones de los datos (log, Box-Cox, normalización) pueden ser necesarias si los datos se desvían significativamente de la normalidad.
- La visualización combinada con estas métricas permite validar los supuestos de muchos algoritmos, especialmente los basados en distribuciones.

Implementación en Python

- Se pueden calcular de forma manual o usando librerías como `scipy.stats` o `pandas`.
- Es recomendable **visualizar los resultados** con histogramas o gráficos de densidad para interpretar mejor los valores de asimetría y curtosis.

2.10 Medidas de forma

Las medidas de forma son de gran utilidad para todas las personas que hagan uso de la estadística, ya que se puede describir la forma que toma una distribución de datos. Las curvas que se utilizan para representar las observaciones de una serie de datos pueden ser **simétricas** o **asimétricas** (sesgadas).

- Las curvas simétricas son aquellas que trazan una línea vertical desde el punto más alto de ella (la cima) hasta el eje horizontal. El área de esta curva será dividida exactamente en dos partes iguales, siendo la parte derecha espejo de la parte izquierda.

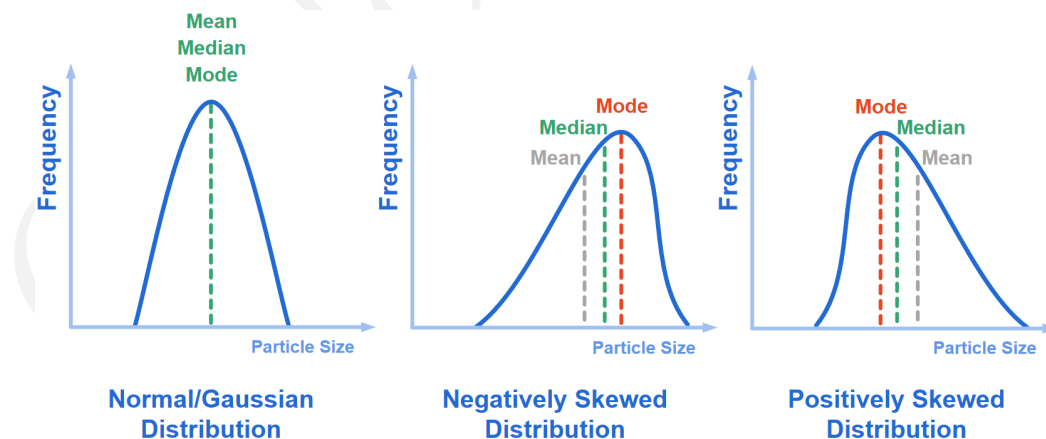


Figura 2.1 – Valores de la característica de recorte en 4

- **Gaussian Distribution:** Cuando la distribución es simétrica (no tiene sesgo), la media, la mediana y la moda se ubicarán en el centro de la distribución. En este caso estas tienen el mismo valor.

$$\mu = Me = Mo \quad (2.5)$$

- **Positively Skewed Distribution (Sesgada hacia la derecha):** Esta curva esta sesgada hacia la derecha o con simetría positiva, lo que se debe que disminuye de manera gradual hacia el extremo superior de la escala. En esta curva, la moda es el punto más alto, la mediana el punto medio, mientras que la media siempre tiende a ubicarse hacia la cola (derecha) de la distribución. Esto se debe a que la media siempre será afectada por los valores extremos. En este caso la media, la mediana y la moda tienen diferentes valores.

$$\mu > Me > Mo \quad (2.6)$$

- **Negatively Skewed Distribution (Sesgada hacia la izquierda):** Esta tiene sesgo a la izquierda o asimetría negativa, ya que disminuye de forma gradual el extremo inferior de la escala. En esta curva, la moda es el punto más alto, la media es el medio, mientras que la mediana siempre tenderá a ubicarse hacia la cola (izquierda) de la distribución.

$$Mo > Me > \mu \quad (2.7)$$

Categorizar y medir correctamente la asimetría permite comprender cómo se distribuyen los valores en torno a la media e influir en la elección de técnicas estadísticas y transformaciones de datos. Por ejemplo, las distribuciones muy sesgadas podrían beneficiarse de técnicas de normalización o escalado para asemejarlas a una distribución normal. Esto ayudaría al rendimiento del modelo.

Para recordar las diferencias entre asimetría positiva y negativa, piénsalo así:

- Si quieres aumentar la media de una distribución, debes añadir a la distribución valores mucho más altos que la media.
- Para bajar la media, debes hacer lo contrario: introducir en la distribución valores mucho más bajos que la media.
- Por tanto, si la mayoría de los valores extremos es superior a la media, la asimetría será positiva porque aumentan la media (Extremos altos dominan → tiran la media hacia arriba → asimetría positiva).
- Si la mayoría de los valores extremos es menor que la media, la asimetría es negativa porque disminuyen la media (Extremos bajos dominan → tiran la media hacia abajo → asimetría negativa.)

2.10.1. Cómo calcular la asimetría en Python

Hay muchas formas de calcular la asimetría, pero la más sencilla es el segundo coeficiente de asimetría de Pearson, también conocido como asimetría media.

$$\text{Skewness} = \frac{3(\mu - \tilde{x})}{\sigma} \quad (2.8)$$

Implementemos la fórmula manualmente en Python:

```

1      import numpy as np
2      import pandas as pd
3      import seaborn as sns
4
5      # Example dataset
6      diamonds = sns.load_dataset("diamonds")
7      diamond_prices = diamonds["price"]
8
9      mean_price = diamond_prices.mean()
10     median_price = diamond_prices.median()
11     std = diamond_prices.std()
12
13     skewness = (3 * (mean_price - median_price)) / std
14     print(
15         f"The Pierson's second skewness score of diamond
16         prices distribution is {skewness:.5f}"
17     )

```

$$\text{Skewness} = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \mu}{\sigma} \right)^3 \quad (2.9)$$

```

1      def moment_based_skew(distribution):
2          n = len(distribution)
3          mean = np.mean(distribution)
4          std = np.std(distribution)
5
6          # Divide the formula into two parts
7          first_part = n / ((n - 1) * (n - 2))
8          second_part = np.sum(((distribution - mean) / std) **
9                               3)
10
11         skewness = first_part * second_part
12
13         return skewness
14
15     moment_based_skew(diamond_prices)
16
17     # Pandas version
18     diamond_prices.skew()
19
20     # SciPy version
21     from scipy.stats import skew
22
23     skew(diamond_prices)

```

Aunque las distintas fórmulas para aproximar la asimetría producen valores numéricos diferentes, estas discrepancias son demasiado pequeñas para ser significativas o modificar la clasificación final de la asimetría. En otras palabras, aunque los métodos empleados hoy utilizan expresiones distintas bajo el capó, todos conducen a resultados muy similares.

Una vez calculada la asimetría, se puede clasificar según los siguientes intervalos:

- **Baja o aproximadamente simétrica:** $a \in [-0,5, 0,5]$
- **Moderadamente sesgada:** $a \in [-1, -0,5) \cup (0,5, 1]$
- **Fuertemente sesgada:** $a \in (-\infty, -1) \cup (1, \infty)$

Interpretación del coeficiente de asimetría (skew)

El coeficiente de asimetría indica la dirección del sesgo de una distribución. Su interpretación es la siguiente:

$$\begin{cases} \text{skew} > 0 & \Rightarrow \text{Sesgo a la derecha (right-skewed)} \\ \text{skew} < 0 & \Rightarrow \text{Sesgo a la izquierda (left-skewed)} \\ \text{skew} = 0 & \Rightarrow \text{Distribución simétrica} \end{cases}$$

Sesgo a la derecha (skew > 0):

- La cola de la distribución se extiende hacia la derecha (valores altos).
- La mayoría de los datos se acumulan en valores pequeños.

Sesgo a la izquierda (skew < 0):

- La cola de la distribución se extiende hacia la izquierda (valores bajos).
- La mayoría de los datos se acumulan en valores grandes.

1. Heurística de Imputación (recom_imputer)

Sea outliers % el porcentaje de valores atípicos y skew el coeficiente de asimetría.

$$\text{Imputer} = \begin{cases} \text{mediana,} & \text{si outliers_ \%} \geq 5 \% \text{ o } |\text{skew}| \geq 1,0, \\ \text{media,} & \text{en caso contrario.} \end{cases}$$

Fundamento. La media es un estimador no robusto frente a outliers y a distribuciones muy asimétricas, mientras que la mediana posee un punto de ruptura alto y representa mejor el centro en presencia de valores extremos o sesgo marcado.

2. Heurística de Escalado (recom_scaler)

$$\text{Scaler} = \begin{cases} \text{RobustScaler,} & \text{si outliers_ \%} \geq 5 \%, \\ \text{StandardScaler,} & \text{si outliers_ \%} < 5 \% \text{ y } |\text{skew}| \leq 0,5, \\ \text{MinMaxScaler,} & \text{en cualquier otro caso.} \end{cases}$$

Fundamento. El **RobustScaler** se basa en la mediana y el rango intercuartílico, lo que le permite mantener estabilidad ante valores atípicos. El **StandardScaler** resulta adecuado para distribuciones aproximadamente simétricas y sin outliers significativos. El **MinMaxScaler** no asume ninguna forma particular de distribución y funciona como opción por defecto cuando las condiciones anteriores no se satisfacen.